ELEMENTI DI BIOINFORMATICA

Documentazione ed Elaborato

Analisi di geni differenzialmente espressi

Adenocarcinoma polmonare vs tessuto polmonare adiacente non tumorale (GEO GSE32863)

Pipeline sviluppata in R / Bioconductor

Banche dati: NCBI GEO, Enrichr (GO, KEGG, Reactome, WikiPathways)

Studente: **Daniele Murgese**

Matricola: **0322500069**

Link Progetto: https://github.com/danielemurgese992/Elementi-Bioinformatica

Anno accademico 2026/2027

Indice

1. Introduzione e obiettivi 4
1.1 Ambiente di esecuzione dei risultati 4
2. Banche dati e dataset scelto 5
2.1 La banca dati NCBI GEO 5
2.2 Scelta del dataset 5
2.3 Caratteristiche del dataset GSE32863 5
2.4 La tecnologia Illumina BeadChip 6
2.5 Altre banche dati utilizzate 6
3. Ambiente di lavoro e pacchetti R 7
3.1 Installazione dell'ambiente ed esecuzione del progetto 8
4. Download e importazione dei dati 12
4.1 Teoria: la classe ExpressionSet 12
4.2 Codice 12
5. Descrizione del dataset e caratteristiche cliniche 14
5.1 Teoria 14
5.2 Codice 14
6. Estrazione dei campioni: casi vs controlli 19
7. Pre-elaborazione (1): trasformazione logaritmica 20
7.1 Teoria 20
7.2 Codice 20
8. Pre-elaborazione (2): filtro sull'intervallo interquartile 22
8.1 Teoria 22
8.2 Codice 22
9. Annotazione: da sonda a gene 24
10. Filtraggio statistico: fold-change, test t e FDR 25
10.1 Il fold-change in scala logaritmica 25
10.2 Scelta del test statistico 25
10.3 Il problema dei test multipli e il False Discovery Rate 25
10.4 Codice 26
11. Volcano plot e selezione dei DEG 28
11.1 Teoria 28
11.2 Codice 28
12. Grafici e ulteriori analisi 30
12.1 Boxplot dei geni più up- e down-regolati 30
12.2 Clustering gerarchico e heatmap 31
12.3 Analisi delle componenti principali (PCA) 34
13. Esportazione dei risultati 37
14. Analisi di arricchimento funzionale con Enrichr 39
14.1 Teoria 39
14.2 Codice 39
14.3 Risultati: geni up-regolati 40
14.4 Risultati: geni down-regolati 43
14.5 Confronto con la letteratura 45
15. Analisi di sensibilità: test t appaiato 47
16. Approfondimento con banche dati biologiche e ricerca bibliografica 48
16.1 Metodologia 48
16.2 Geni candidati noti in letteratura 48
16.3 Schede dei geni principali ottenuti 49
16.4 Collegamento con lo studio originale 49
17. Discussione e limiti 50

18. Conclusioni 51
Appendice A – Script R completo 52
Appendice B – File prodotti dallo script 60
Appendice C – Ambiente software (sessionInfo) 61
Bibliografia.....62

1. Introduzione e obiettivi

L'obiettivo dell'elaborato è identificare l'elenco dei geni **differenzialmente espressi** (DEG, *Differentially Expressed Genes*) tra campioni di tipo "case" e campioni di tipo "control" e caratterizzarli dal punto di vista funzionale. Nel presente lavoro i casi sono campioni di **adenocarcinoma polmonare** (LUAD, *lung adenocarcinoma*) e i controlli sono campioni di **tessuto polmonare adiacente istologicamente normale** prelevati dagli stessi pazienti (*matched-normal*).

L'intera pipeline è implementata in linguaggio **R** con pacchetti del progetto **Bioconductor** e di CRAN, ed è organizzata nelle seguenti fasi, che rispecchiano la traccia dell'esercitazione:

1. Download dei dati di espressione dalla banca dati NCBI GEO e importazione in R.
2. Descrizione del dataset e delle caratteristiche cliniche della coorte; definizione dei gruppi casi/controlli.
3. Pre-elaborazione: trasformazione log2(x + 1) e filtro non specifico basato sull'intervallo interquartile (IQR).
4. Filtraggio statistico: log fold-change, test t, correzione dei p-value con False Discovery Rate, volcano plot e soglie.
5. Grafici e analisi esplorative: boxplot dei geni più up/down-regolati, clustering gerarchico con heatmap, PCA.
6. Esportazione dei risultati in un file .txt.
7. Analisi di arricchimento funzionale con Enrichr.
8. Approfondimento mediante banche dati biologiche e ricerca bibliografica.

Per ogni fase il documento riporta: (i) la **parte teorica** necessaria a motivare le scelte, (ii) lo **script R** corrispondente e (iii) la **spiegazione del codice**. Lo script completo è riportato in Appendice A ed è fornito anche come file separato `analisi_DEG_GSE32863.R`.

1.1 Ambiente di esecuzione dei risultati

Tutti i risultati, le tabelle e le figure riportati nel documento sono stati ottenuti eseguendo lo script su Windows 11 (64 bit) con **R 4.6.1** e i pacchetti GEOquery 2.80.0, Biobase 2.72.0, ggplot2 4.0.3, pheatmap 1.0.13, matrixStats 1.5.0 ed enrichR 3.4 (elenco completo in Appendice C). I riquadri azzurri **"Risultato"** riportano l'output della console (file `log_console.txt`); le figure coincidono con i file PNG generati nella cartella `output/`.

2. Banche dati e dataset scelto

2.1 La banca dati NCBI GEO

Gene Expression Omnibus (GEO) è il repository pubblico del *National Center for Biotechnology Information* (NCBI) che archivia dati di genomica funzionale ad alta processività: microarray di espressione, RNA-seq, ChIP-seq, metilazione del DNA e altro. GEO è conforme alle linee guida **MIAME** (*Minimum Information About a Microarray Experiment*), che impongono di depositare insieme ai dati le informazioni minime necessarie per interpretarli e riprodurli.

I dati in GEO sono organizzati in quattro tipi di record, identificati da un prefisso:

Prefisso	Record	Contenuto
GPL	Platform	Descrizione della piattaforma: elenco delle sonde e loro annotazione (simbolo genico, Entrez ID, RefSeq…). Nel nostro caso GPL6884.
GSM	Sample	Singolo campione: caratteristiche biologiche/cliniche, protocolli e tabella dei valori di espressione.
GSE	Series	Insieme di campioni che costituiscono uno studio, con descrizione del disegno sperimentale. Nel nostro caso GSE32863.
GDS	DataSet	Versione curata da GEO di una Series, con campioni confrontabili e valori omogenei (non disponibile per tutte le serie).

Per ogni Series GEO mette a disposizione il file **Series Matrix** (`GSE…_series_matrix.txt.gz`), una tabella tab-delimitata in cui le righe sono le sonde, le colonne sono i campioni e le intestazioni (righe che iniziano con `!`) contengono i metadati. È il formato utilizzato in questo lavoro, perché contiene i dati già processati e normalizzati dagli autori e può essere letto direttamente con il pacchetto `GEOquery`.

2.2 Scelta del dataset

Tra i dataset proposti dalla traccia (adenocarcinoma polmonare, carcinoma epatocellulare, Alzheimer/MCI, Parkinson, morbo di Crohn, artrite reumatoide) è stato scelto **GSE32863** per i seguenti motivi:

- **Numerosità adeguata e bilanciata**: 58 tumori e 58 controlli; entrambi i gruppi superano la soglia n > 30 indicata dalla traccia per applicare il teorema del limite centrale e il test t.
- **Controlli matched-normal**: tumore e tessuto normale provengono dagli stessi pazienti, il che riduce i fattori confondenti individuali (genetica, età, abitudine al fumo) e permette anche un'analisi appaiata di conferma.
- **Unica piattaforma** (GPL6884) e dati già normalizzati, quindi nessun problema di integrazione tra piattaforme diverse.
- **Letteratura di riferimento ampia**: l'adenocarcinoma polmonare è il sottotipo istologico più frequente di carcinoma polmonare ed è ben caratterizzato (es. TCGA-LUAD), il che facilita la validazione dei risultati.

2.3 Caratteristiche del dataset GSE32863

Campo	Valore
Accession	GSE32863 (SubSeries della SuperSeries GSE32867)
Titolo	Gene expression analysis of lung adenocarcinoma and matched adjacent non-tumor lung tissue

Campo	Valore
Organismo	Homo sapiens
Tipo di esperimento	Expression profiling by array
Piattaforma	GPL6884 – Illumina HumanWG-6 v3.0 expression beadchip
Campioni	116: 58 adenocarcinomi polmonari + 58 tessuti polmonari adiacenti non tumorali (fresh frozen, macrodissezionati)
Coppie complete	57 pazienti con entrambi i campioni (per un paziente è disponibile solo il tumore e per un altro solo il normale)
Pubblicazione	Selamat SA et al., Genome Research 2012 (PMID 22613842)
Data pubblicazione	21 marzo 2012
Link	https://www.ncbi.nlm.nih.gov/geo/query/acc.cgi?acc=GSE32863

La traccia indica 60 casi e 60 controlli: il numero corrisponde ai campioni profilati nello studio originale, mentre la Series pubblicata su GEO contiene 58 + 58 campioni, come riportato nel campo *Overall design*. L'analisi fa riferimento ai 116 campioni effettivamente disponibili.

Lo studio originale (Selamat et al., 2012) aveva come obiettivo l'integrazione dei profili di metilazione del DNA con quelli di espressione genica, per identificare geni la cui espressione nell'adenocarcinoma polmonare è regolata epigeneticamente. In questo elaborato si utilizza esclusivamente la componente di espressione.

2.4 La tecnologia Illumina BeadChip

Il chip **Illumina HumanWG-6 v3.0** consente di profilare 6 campioni per vetrino, con circa 48.000 sonde per campione. Ogni sonda è un oligonucleotide di 50 basi legato a microsfere (*beads*) depositate casualmente in micro-pozzetti; ciascun tipo di sonda è presente in circa 30 copie, e il valore di espressione riportato è un riassunto robusto delle intensità di fluorescenza delle copie. L'RNA del campione viene retrotrascritto, amplificato, marcato con biotina e ibridato sul chip; l'intensità del segnale è proporzionale all'abbondanza del trascritto.

Due conseguenze pratiche per l'analisi: (1) dopo la sottrazione del background alcune sonde poco espresse possono avere **valori negativi o prossimi allo zero**, che vanno gestiti prima del logaritmo; (2) più sonde possono riferirsi allo **stesso gene**, per cui è necessario un passo di "collasso" da sonda a gene.

2.5 Altre banche dati utilizzate

Banca dati / strumento	Uso nell'elaborato
NCBI GEO	Fonte dei dati di espressione e dei metadati clinici (GSE32863) e dell'annotazione della piattaforma (GPL6884).
Enrichr (Ma'ayan Lab)	Analisi di arricchimento funzionale delle liste di DEG up- e down-regolati.
Gene Ontology (GO)	Libreria di Enrichr "GO*Biological*Process_2023": processi biologici.
KEGG	Libreria "KEGG*2021*Human": pathway metabolici e di segnalazione.
Reactome / WikiPathways	Librerie di pathway curati usate come conferma indipendente.
NCBI Gene, GeneCards, UniProt	Funzione dei singoli geni di interesse.
Human Protein Atlas, GEPIA2	Validazione su dati indipendenti (TCGA-LUAD) e valore prognostico.
STRING	Rete di interazioni proteina-proteina tra i DEG.
PubMed	Ricerca bibliografica sui geni e i pathway emersi.

3. Ambiente di lavoro e pacchetti R

L'analisi richiede R ≥ 4.2. I pacchetti utilizzati e il loro ruolo sono riassunti nella tabella seguente.

Pacchetto	Repository	Funzione
GEOquery	Bioconductor	Download e parsing dei file GEO; restituisce oggetti ExpressionSet.
Biobase	Bioconductor	Classe ExpressionSet e accessori exprs(), pData(), fData().
matrixStats	CRAN	Statistiche per riga ottimizzate in C (rowIQRs).
ggplot2, ggrepel	CRAN	Grafici basati sulla grammatica dei grafici; etichette non sovrapposte.
pheatmap, RColorBrewer	CRAN	Heatmap con clustering gerarchico e palette di colori.
dplyr, tidyr, gridExtra	CRAN	Manipolazione dei dati e composizione di più grafici.
enrichR	CRAN	Interfaccia R alle API web di Enrichr.

Script R – Setup

```
## ---- 0. SETUP: pacchetti, parametri, cartelle ---------------------------
# Installazione (da eseguire una sola volta):
# if (!requireNamespace("BiocManager", quietly = TRUE))
#   install.packages("BiocManager")
# BiocManager::install(c("GEOquery", "Biobase"))
# install.packages(c("ggplot2", "ggrepel", "pheatmap", "RColorBrewer",
#                    "matrixStats", "dplyr", "tidyr", "gridExtra", "enrichR"))

suppressPackageStartupMessages({
  library(GEOquery)      # download e parsing dei dati GEO
  library(Biobase)       # classe ExpressionSet (exprs, pData, fData)
  library(matrixStats)   # statistiche per riga veloci (rowIQRs, ...)
  library(ggplot2)       # grafici
  library(ggrepel)       # etichette non sovrapposte nel volcano plot
  library(pheatmap)      # heatmap con clustering gerarchico
  library(RColorBrewer)  # palette di colori
  library(dplyr)         # manipolazione tabelle
  library(tidyr)         # formato long/wide
  library(gridExtra)     # composizione di piu grafici
})

set.seed(123)                # riproducibilita
GSE_ID   <- "GSE32863"       # identificativo della serie GEO
DATA_DIR <- "data"           # cartella per i file scaricati
OUT_DIR  <- "output"         # cartella per grafici e risultati
dir.create(DATA_DIR, showWarnings = FALSE)
dir.create(OUT_DIR,  showWarnings = FALSE)

IQR_PERC <- 0.10   # percentile della distribuzione IQR sotto cui scartare
FC_THR   <- 2      # soglia di fold-change lineare -> |logFC| >= log2(2) = 1
ALPHA    <- 0.05   # soglia sul p-value aggiustato (FDR)
options(timeout = 1200)       # download lenti

# funzione di utilita per salvare i grafici ggplot in PNG
save_plot <- function(p, file, w = 8, h = 6) {
```

```
  ggsave(file.path(OUT_DIR, file), p, width = w, height = h, dpi = 150)
}
```

Spiegazione del codice

- Le righe commentate con `install.packages` / `BiocManager::install` vanno eseguite una sola volta: i pacchetti Bioconductor si installano tramite `BiocManager`, quelli CRAN con `install.packages`.
- `suppressPackageStartupMessages` carica le librerie senza stampare i messaggi di avvio.
- `set.seed(123)` rende riproducibili le parti casuali (posizione dei punti nel jitter dei boxplot).
- I parametri dell'analisi sono definiti **una sola volta in testa allo script**: percentile IQR (`IQR_PERC`), soglia di fold-change (`FC_THR`) e livello di significatività (`ALPHA`). Modificarli consente di ripetere l'analisi con soglie diverse senza toccare il resto del codice.
- `options(timeout = 1200)` aumenta il timeout dei download, utile per i file GEO di grandi dimensioni.
- `save_plot` è una funzione di utilità che salva i grafici ggplot2 in PNG a 150 dpi nella cartella `output/`.

3.1 Installazione dell'ambiente ed esecuzione del progetto

Questa sezione descrive passo per passo come predisporre l'ambiente di lavoro e riprodurre l'intera analisi su un personal computer. La procedura è stata seguita per ottenere i risultati riportati nel documento (Windows 11, R 4.6.1). Non è necessario scaricare manualmente alcun file di dati: al primo avvio lo script scarica da GEO sia la matrice di espressione sia l'annotazione della piattaforma GPL6884.

Requisiti: un computer Windows, macOS o Linux; una **connessione a internet attiva** durante l'esecuzione (necessaria per il download da NCBI GEO e per l'analisi con Enrichr); alcune centinaia di MB di spazio libero su disco.

Installazione di R e RStudio

1. **R** (versione 4.3 o superiore) si scarica da https://cran.r-project.org/. Su Windows: "Download R for Windows" → "base" → eseguire l'installer con le opzioni predefinite. Su macOS: "Download R for macOS" e scegliere il file `.pkg` adatto al processore (Apple Silicon o Intel).
2. **RStudio Desktop** (gratuito) si scarica da https://posit.co/download/rstudio-desktop/ e va installato **dopo** R, che RStudio utilizza come motore di calcolo.
3. Su Windows non serve installare Rtools: i pacchetti necessari vengono scaricati in versione già compilata.

RStudio è un ambiente di sviluppo integrato (IDE) che organizza il lavoro in quattro pannelli: l'**editor** degli script (in alto a sinistra), la **Console** in cui R esegue i comandi (in basso a sinistra), l'**Environment** con gli oggetti in memoria (in alto a destra) e il pannello **Files/Plots/Packages** (in basso a destra).

Preparazione della cartella di progetto

Si crea una cartella di lavoro (ad esempio `C:\bioinfo\elaborato` su Windows o `~/bioinfo/elaborato` su macOS) e vi si copia il file `analisi_DEG_GSE32863.R`. Le sottocartelle `data/` e `output/` vengono create dallo script durante l'esecuzione. A fine analisi la struttura è la seguente:

Struttura della cartella di progetto

```
elaborato/
├── analisi_DEG_GSE32863.R      # script dell'analisi
├── log_console.txt             # output della console (creato con sink)
├── data/                       # file scaricati da GEO (Series Matrix, GPL6884)
└── output/                     # grafici PNG e tabelle .txt dei risultati
```

Installazione dei pacchetti R

I pacchetti si installano **una sola volta**, incollando i comandi seguenti nella **Console** di RStudio. I comandi vanno eseguiti su righe separate (oppure separati da `;`): scritti di seguito sulla stessa riga producono l'errore di sintassi `unexpected symbol` e nessun pacchetto viene installato.

Console R – Installazione dei pacchetti

```
install.packages("BiocManager")
BiocManager::install(c("GEOquery", "Biobase"), update = FALSE, ask = FALSE)
install.packages(c("ggplot2", "ggrepel", "pheatmap", "RColorBrewer",
                   "matrixStats", "dplyr", "tidyr", "gridExtra", "enrichR"))
```

GEOquery e Biobase fanno parte di **Bioconductor** e si installano tramite `BiocManager`; gli altri pacchetti provengono da **CRAN** e si installano con `install.packages`. Durante l'installazione R può porre alcune domande:

- scelta del **mirror CRAN**: un server qualsiasi (ad esempio "Italy" o "0-Cloud");
- "*Would you like to use a personal library instead?*": rispondere `yes` (anche alla domanda successiva), così i pacchetti vengono installati nella cartella dell'utente;
- "*Do you want to install from sources the packages which need compilation?*": rispondere `no`, per usare le versioni già compilate.

L'installazione richiede alcuni minuti; è terminata quando nella Console scompare il simbolo rosso di stop. Per verificare che tutti i pacchetti siano presenti:

Console R – Verifica dell'installazione

```
pk <- c("GEOquery", "Biobase", "ggplot2", "ggrepel", "pheatmap", "RColorBrewer",
        "matrixStats", "dplyr", "tidyr", "gridExtra", "enrichR")
pk[!pk %in% rownames(installed.packages())]   # character(0) = tutto installato
```

Se il risultato è `character(0)` tutti i pacchetti sono installati; in caso contrario vengono elencati quelli mancanti, da reinstallare.

Impostazione della cartella di lavoro

R legge e scrive i file a partire dalla **working directory** (cartella di lavoro). Questa deve coincidere con la cartella che contiene lo script, perché il comando di esecuzione richiama lo script con un percorso relativo e le cartelle `data/` e `output/` vengono create proprio lì. In RStudio la si può impostare in tre modi:

- dal menu: aprire lo script nell'editor (*File → Open File…*) e scegliere *Session → Set Working Directory → To Source File Location*;
- dalla Console, con `setwd("C:/bioinfo/elaborato")`: su Windows il percorso va scritto con `/` al posto di `\`;
- dal pannello *Files*: entrare nella cartella e scegliere *More → Set As Working Directory*.

Il comando `setwd()` non stampa alcun messaggio se va a buon fine; in caso di percorso errato restituisce l'errore `cannot change working directory`. Per verificare l'impostazione:

Console R – Verifica della working directory

```
getwd()        # deve restituire la cartella del progetto
list.files()   # deve elencare "analisi_DEG_GSE32863.R"
```

La working directory si azzera a ogni riavvio di RStudio. Per non doverla reimpostare si può creare un **progetto RStudio** (*File → New Project → Existing Directory*, selezionando la cartella dell'elaborato): aprendo il progetto, la cartella di lavoro viene impostata automaticamente.

Esecuzione dell'analisi

Metodo A – RStudio (consigliato). Con la working directory impostata, si eseguono nella Console i comandi seguenti:

Console R – Esecuzione dello script

```
sink("log_console.txt", split = TRUE)
source("analisi_DEG_GSE32863.R", echo = TRUE, max.deparse.length = Inf)
sink()
```

- `sink(…, split = TRUE)` duplica l'output della Console nel file `log_console.txt`, conservando la visualizzazione a schermo: il log documenta l'esecuzione e contiene i valori numerici riportati nel documento.
- `source(…, echo = TRUE)` esegue l'intero script mostrando ogni comando prima del suo output; `max.deparse.length = Inf` evita che i comandi lunghi vengano troncati nel log.
- Il secondo `sink()` chiude il file di log. Va eseguito anche se lo script si interrompe per un errore, altrimenti l'output successivo della Console continuerebbe a essere scritto nel file.

Metodo B – Terminale. In alternativa si apre un terminale (Prompt dei comandi o PowerShell su Windows, Terminale su macOS) nella cartella del progetto e si esegue:

Terminale – Esecuzione con Rscript

```
Rscript analisi_DEG_GSE32863.R > log_console.txt 2>&1
```

`Rscript` esegue lo script in modo non interattivo; `> log_console.txt 2>&1` reindirizza nel file sia l'output standard sia i messaggi di errore. Su Windows, se il comando `Rscript` non viene riconosciuto,

occorre indicare il percorso completo dell'eseguibile, ad esempio `"C:\Program Files\R\R-4.6.1\bin\Rscript.exe"`.

Durata dell'esecuzione e verifica dei risultati

Al primo avvio lo script scarica da GEO alcune decine di MB di dati: l'esecuzione completa richiede quindi alcuni minuti e non va interrotta. Le esecuzioni successive riutilizzano i file presenti in `data/` e sono più rapide. L'analisi è completata correttamente quando nella Console (e in fondo al log) compare il messaggio "**Analisi completata. Risultati in: …**". La cartella `output/` contiene allora tutti i file elencati in Appendice B; il file `sessionInfo.txt` registra le versioni di R e dei pacchetti utilizzate.

Risoluzione dei problemi più comuni

Problema	Causa	Soluzione
Barra gialla in RStudio che chiede di installare pacchetti anche dopo l'installazione	RStudio analizza le chiamate `library()` all'apertura dello script e non aggiorna l'avviso	Verificare con il comando di controllo dell'installazione; chiudere e riaprire lo script o riavviare la sessione (*Session → Restart R*). Non usare il pulsante "Install" della barra per GEOquery e Biobase, che sono pacchetti Bioconductor.
Errore `unexpected symbol` durante l'installazione	Più comandi incollati sulla stessa riga	Incollare un comando per riga oppure separarli con `;`.
Errore `cannot change working directory`	Il percorso passato a `setwd()` non esiste o contiene `\`	Controllare il percorso e usare `/` come separatore.
Errore `argomento 'pattern' non valido` nella sezione Enrichr	La funzione `setEnrichrSite` richiamata senza caricare il pacchetto (versione iniziale dello script)	Già corretto nello script allegato con `library(enrichR)` prima di `setEnrichrSite`; eseguire `sink()` e rilanciare l'analisi.
Messaggi "EnrichR UP/DOWN: …" nel log	Nome di una libreria Enrichr cambiato o servizio temporaneamente non raggiungibile	Consultare i nomi disponibili con `enrichR::listEnrichrDbs()` e aggiornare il vettore `dbs`; in alternativa caricare `lista_UP.txt` e `lista_DOWN.txt` sul sito di Enrichr.
Download da GEO interrotto o in timeout	Connessione lenta o instabile	Rilanciare lo script: `options(timeout = 1200)` è già impostato; eventualmente cancellare i file incompleti in `data/`.
`Rscript` non riconosciuto dal terminale	La cartella di R non è nella variabile PATH	Usare il percorso completo di `Rscript.exe` oppure il Metodo A.

4. Download e importazione dei dati

4.1 Teoria: la classe ExpressionSet

`getGEO` restituisce una lista di oggetti **ExpressionSet** (uno per piattaforma). L'ExpressionSet è la struttura dati standard di Bioconductor per i dati di espressione e raccoglie in un unico oggetto tre componenti coerenti tra loro:

- `assayData`, accessibile con `exprs()`: la matrice di espressione G × N (G sonde, N campioni).
- `phenoData`, accessibile con `pData()`: un data frame N righe con i metadati dei campioni (titolo, tessuto, caratteristiche cliniche).
- `featureData`, accessibile con `fData()`: un data frame G righe con l'annotazione delle sonde, ricavata dalla piattaforma GPL.

Il vantaggio è che sottoinsiemi di campioni o sonde mantengono automaticamente allineati dati e metadati.

4.2 Codice

Script R – Download e importazione

```
## ---- 1. DOWNLOAD E IMPORTAZIONE DEI DATI --------------------------------
# getGEO scarica il file "Series Matrix" (dati gia normalizzati dagli autori)
# e, con getGPL = TRUE, la tabella di annotazione della piattaforma GPL6884.
gse_list <- getGEO(GSE_ID, GSEMatrix = TRUE, getGPL = TRUE,
                   destdir = DATA_DIR)
length(gse_list)              # una sola piattaforma -> lista di lunghezza 1
gse  <- gse_list[[1]]         # oggetto ExpressionSet

expr <- exprs(gse)            # matrice sonde x campioni
pd   <- pData(gse)            # metadati dei campioni (phenoData)
fd   <- fData(gse)            # annotazione delle sonde (featureData)

cat("Sonde:", nrow(expr), " - Campioni:", ncol(expr), "\n")
print(expr[1:5, 1:4])         # prime righe/colonne della matrice
print(summary(as.vector(expr)))
print(colnames(fd))           # colonne disponibili per l'annotazione
```

Spiegazione del codice

- `getGEO(GSE_ID, GSEMatrix = TRUE, getGPL = TRUE, destdir = DATA_DIR)` scarica il Series Matrix e la tabella GPL6884 nella cartella `data/`; alle esecuzioni successive i file vengono riletti da disco senza nuovo download.
- Poiché GSE32863 usa una sola piattaforma, la lista ha lunghezza 1 e si estrae il primo elemento.
- `exprs`, `pData` e `fData` estraggono le tre componenti descritte sopra.
- `summary(as.vector(expr))` mostra la distribuzione globale dei valori: serve a capire se i dati sono in scala lineare o già logaritmica (decisione presa nella sezione 7).
- `colnames(fd)` elenca le colonne di annotazione disponibili, tra cui quella con il simbolo genico (`Symbol`) usata nella sezione 9.

Risultato: il Series Matrix contiene **48.803 sonde** e **116 campioni**. La tabella di annotazione GPL6884 presenta 28 colonne, tra cui `Symbol`, `ILMN_Gene`, `Entrez_Gene_ID`, `Chromosome` e `Definition`. Il riepilogo

> dei valori (minimo 6,34; mediana 6,97; media 7,57; massimo 16,05) indica che i dati depositati sono **già in scala log2**: nessun valore supera 17 e il minimo corrisponde al livello del rumore di fondo del chip.

Sonda	GSM813411 (N)	GSM813412 (T)	GSM813413 (N)	GSM813414 (T)
ILMN_1343291	15,111	15,394	15,661	15,889
ILMN_1343295	11,555	12,580	12,499	13,576
ILMN_1651199	6,895	6,788	6,843	6,871
ILMN_1651209	7,019	7,101	6,814	6,822
ILMN_1651210	6,974	6,894	6,938	6,686

Tabella – Prime righe e colonne della matrice di espressione (N = normale, T = tumore).

5. Descrizione del dataset e caratteristiche cliniche

5.1 Teoria

Prima di ogni analisi differenziale è necessario descrivere la coorte: la distribuzione di variabili come età, sesso, abitudine al fumo e stadio tumorale permette di valutare la rappresentatività del campione e di individuare possibili **fattori confondenti**. Nel caso di un disegno *matched-normal* le caratteristiche demografiche sono identiche tra i due gruppi per costruzione (stesso paziente), quindi i confondenti individuali sono controllati dal disegno; restano rilevanti come covariate per analisi di sottogruppo.

In GEO le caratteristiche dei campioni sono riportate nei campi `characteristics_ch1` nella forma "chiave: valore". `GEOquery` le converte automaticamente in colonne di `pData` con suffisso `:ch1`.

5.2 Codice

Script R – Caratteristiche cliniche

```
## ---- 2. DESCRIZIONE DEL DATASET E CARATTERISTICHE CLINICHE -----------------
# Le caratteristiche cliniche sono nelle colonne di pData che terminano
# con ":ch1" (derivano dai campi "characteristics_ch1" di GEO).
clin_cols <- grep(":ch1$", colnames(pd), value = TRUE)
clin <- pd[, clin_cols, drop = FALSE]
colnames(clin) <- make.names(sub(":ch1$", "", clin_cols))

# Gruppo (Tumor / Normal) e identificativo del paziente ricavati dal titolo:
# "Lung adenocarcinoma 05L12_T (expression)"
# "Adjacent non-tumor lung tissue 05L12_N (expression)"
clin$group   <- ifelse(grepl("non-tumor", pd$title, ignore.case = TRUE),
                       "Normal", "Tumor")
clin$patient <- sub("^.* (\\S+)_[TN] \\(expression\\)$", "\\1", pd$title)
rownames(clin) <- rownames(pd)

print(table(clin$group))
str(clin)

# Le variabili cliniche si riferiscono al paziente: per non contarle due
# volte si usano solo i campioni tumorali (uno per paziente).
clin_pat <- clin[clin$group == "Tumor", ]

plot_var <- function(df, v) {
  x <- df[[v]]
  xn <- suppressWarnings(as.numeric(as.character(x)))
  if (sum(!is.na(xn)) > 0.8 * length(x) && length(unique(xn)) > 10) {
    # variabile numerica continua (es. eta) -> istogramma
    ggplot(data.frame(val = xn), aes(val)) +
      geom_histogram(bins = 15, fill = "#2E75B6", colour = "white") +
      labs(title = v, x = v, y = "N. pazienti") + theme_bw()
  } else {
    # variabile categorica -> barplot delle frequenze
    ggplot(data.frame(val = factor(x)), aes(val)) +
      geom_bar(fill = "#C55A11") +
      geom_text(stat = "count", aes(label = after_stat(count)), vjust = -0.3) +
      labs(title = v, x = NULL, y = "N. pazienti") + theme_bw() +
      theme(axis.text.x = element_text(angle = 30, hjust = 1))
  }
```

```
}
# variabili da rappresentare: escludo identificativi e colonne costanti
vars <- setdiff(colnames(clin_pat), c("group", "patient"))
vars <- vars[sapply(clin_pat[vars], function(z) {
  k <- length(unique(z)); k > 1 && k < nrow(clin_pat) })]
vars <- unique(c(vars, grep("age", colnames(clin_pat), value = TRUE,
                            ignore.case = TRUE)))
plots <- lapply(vars, function(v) plot_var(clin_pat, v))
if (length(plots) > 0) {
  g <- arrangeGrob(grobs = plots, ncol = 2)
  ggsave(file.path(OUT_DIR, "01_caratteristiche_cliniche.png"), g,
         width = 10, height = 3.5 * ceiling(length(plots) / 2), dpi = 150)
}
# barplot della numerosita dei gruppi
p_grp <- ggplot(clin, aes(group, fill = group)) + geom_bar() +
  geom_text(stat = "count", aes(label = after_stat(count)), vjust = -0.3) +
  scale_fill_manual(values = c(Normal = "#2E75B6", Tumor = "#C00000")) +
  labs(title = "Campioni per gruppo", x = NULL, y = "N. campioni") +
  theme_bw() + theme(legend.position = "none")
save_plot(p_grp, "02_campioni_per_gruppo.png", 5, 4)
```

Spiegazione del codice

- `grep(":ch1$", …)` seleziona tutte le colonne cliniche, indipendentemente dai nomi esatti delle variabili; `make.names` le rende nomi R validi (es. `smoking status` → `smoking.status`).
- Il **gruppo** si ricava dal titolo del campione: i controlli contengono la stringa “non-tumor”. L’**ID paziente** si estrae con un’espressione regolare che cattura il codice prima del suffisso `_T`/`_N` (es. `05L12`).
- Le variabili cliniche sono riferite al paziente: per non contare ogni paziente due volte, i grafici clinici usano solo i 58 campioni tumorali (`clin_pat`).
- `plot_var` sceglie automaticamente il grafico: **istogramma** per variabili numeriche continue (es. età), **barplot** delle frequenze per variabili categoriche (sesso, fumo, stadio). Le colonne costanti o con un valore diverso per ogni riga (identificativi) vengono escluse.
- `arrangeGrob` compone tutti i grafici in un’unica figura a due colonne; un secondo grafico mostra la numerosità dei due gruppi.

Risultato: `pData` contiene 21 variabili annotate, oltre alle due derivate (`group`, `patient`). Si distinguono **variabili cliniche e molecolari** (età, sesso, etnia, abitudine al fumo, pack-years, stadio, recidiva, stato mutazionale di EGFR, KRAS e LKB1) e **variabili tecniche** (batch di ibridazione, numero e striscia del BeadChip, piastra di conversione al bisolfito, fonte del tessuto, misura di metilazione Alu). Lo script le rappresenta tutte; nel documento sono state separate in due figure per leggibilità.

Descrizione della coorte (58 pazienti, un campione tumorale per paziente):

Variabile	Distribuzione
Età	da circa 38 a 88 anni; la maggior parte dei pazienti tra 60 e 75 anni, con moda attorno ai 72 anni
Sesso	Female 45 (78%), Male 13 (22%)
Etnia	White 36, Asian 22
Abitudine al fumo	Current 29, Never 29 (coorte bilanciata fumatori/mai fumatori)
Pack-years	0 per 29 pazienti (i non fumatori); per i fumatori valori prevalentemente tra 20 e 60
Stadio	IA 16, IB 18, IIA 9, IIB 2, IIIA 12, IV 1 (59% in stadio I)

Variabile	Distribuzione
Recidiva	No 25, Yes 19, dato non disponibile (codificato “0”) 14
EGFR	mutato 17, wild-type 41
KRAS	mutato 22, wild-type 36; tra i mutati prevale G12C (7), seguito da G12D e G12V (5 ciascuno)
LKB1 (STK11)	mutato 8, wild-type 48, non disponibile 2

La coorte è dunque composta prevalentemente da donne, con tumori in stadio precoce e una proporzione elevata di non fumatori: è coerente con l’obiettivo dello studio originale di confrontare fumatori e mai fumatori, ma non è rappresentativa della popolazione generale dei pazienti con LUAD, in cui prevalgono gli uomini e i fumatori. Le mutazioni di EGFR e KRAS risultano quasi mutuamente esclusive, come atteso per i driver oncogenici del LUAD.

Tra le variabili tecniche è rilevante il **batch** (Batch 1: 23 pazienti, Batch 2: 35). Dalle prime righe di `str(clin)` si osserva che tumore e normale dello stesso paziente appartengono allo stesso batch (es. 05L12: entrambi Batch 2): il batch è quindi bilanciato tra i gruppi e non si confonde con il confronto Tumor vs Normal.

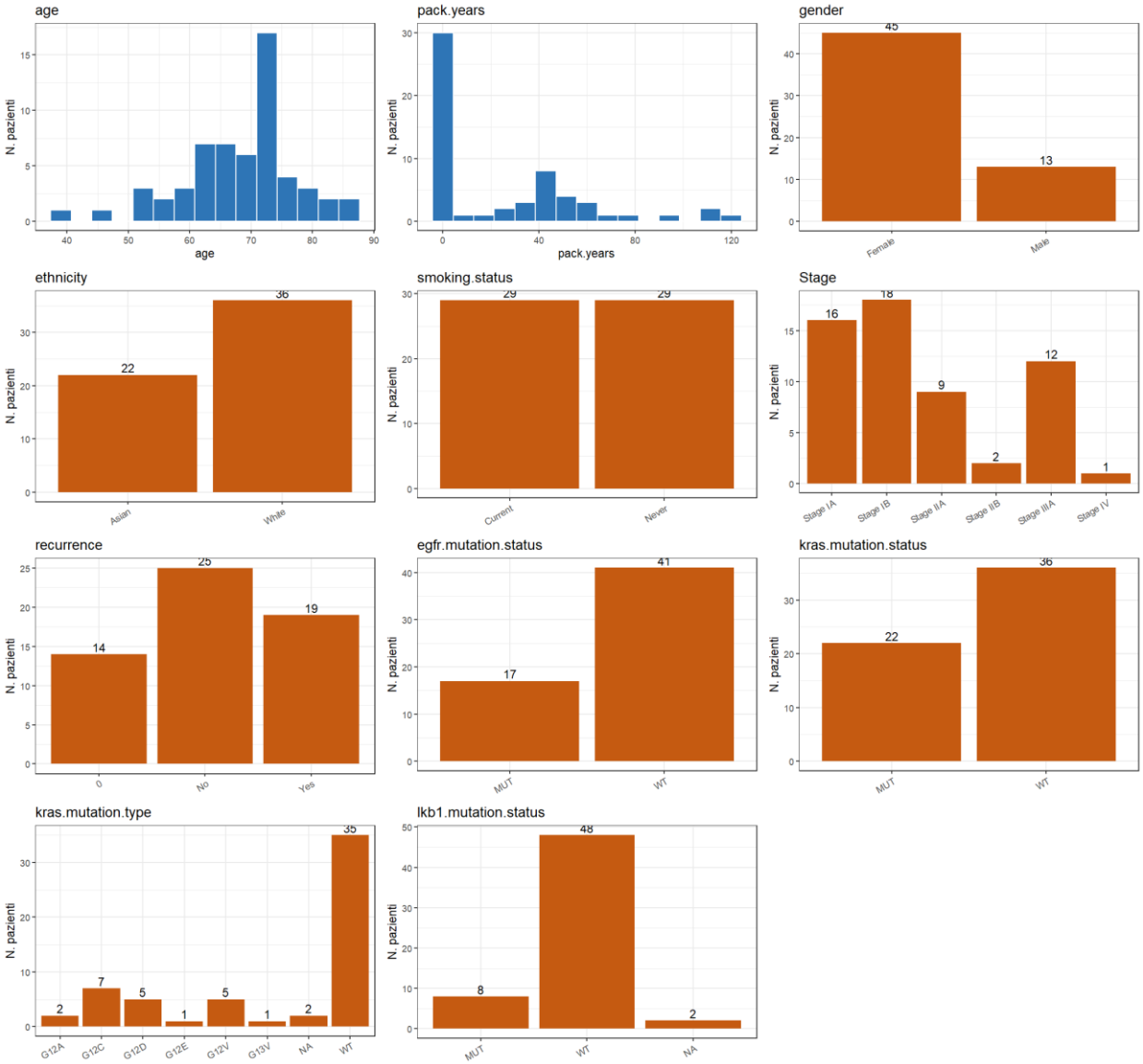


Figura 1 – *Variabili cliniche e molecolari dei 58 pazienti (un campione per paziente) (file: 01_caratteristiche_cliniche.png)*

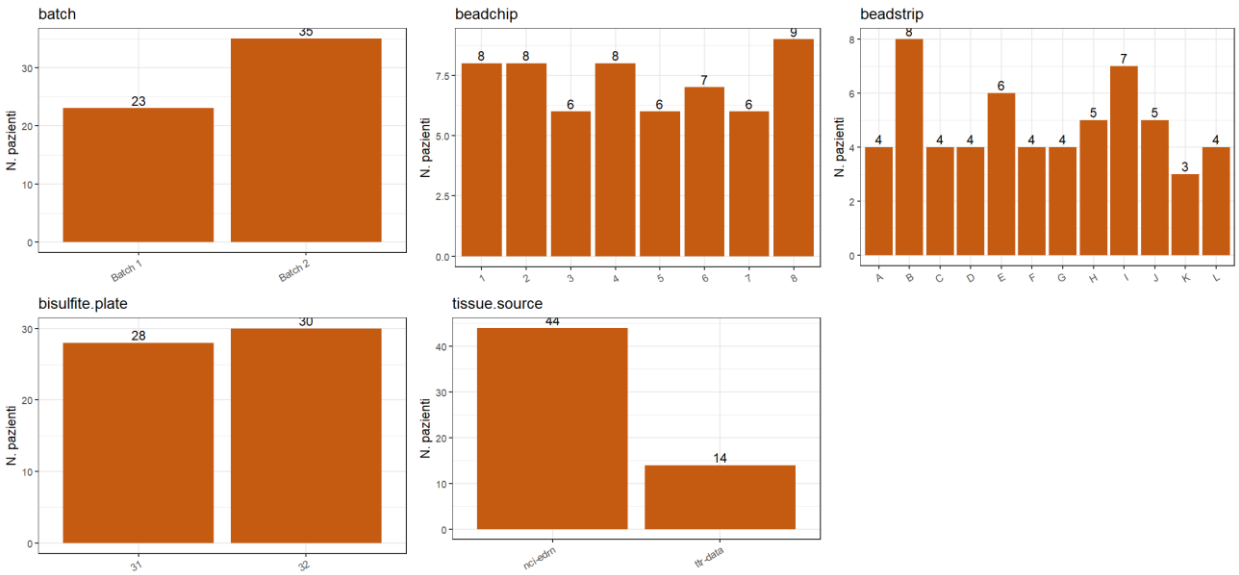


Figura 2 – *Variabili tecniche presenti nei metadati GEO (file: 01_caratteristiche_cliniche.png)*

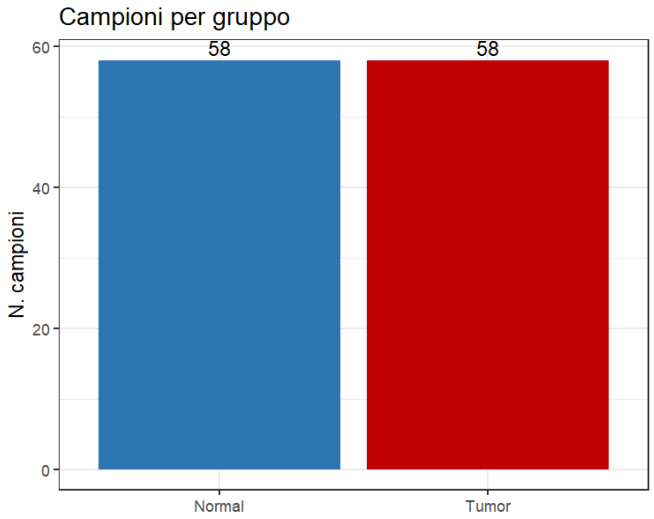


Figura 3 – *Numerosità dei gruppi Tumor e Normal (file: 02_campioni_per_gruppo.png)*

6. Estrazione dei campioni: casi vs controlli

Il confronto è **Tumor (casi) vs Normal (controlli)**. Il fattore `grp` ha come livello di riferimento `Normal`, quindi un logFC positivo indicherà un gene più espresso nel tumore (up-regolato) e uno negativo un gene meno espresso nel tumore (down-regolato).

Script R – Definizione dei gruppi

```
## ---- 3. ESTRAZIONE DEI CAMPIONI DA CONFRONTARE: CASI vs CONTROLLI ----------
grp  <- factor(clin$group, levels = c("Normal", "Tumor"))  # Normal = riferimento
case <- grp == "Tumor"     # vettore logico dei casi
ctrl <- grp == "Normal"    # vettore logico dei controlli
cat("Casi (Tumor):", sum(case), " - Controlli (Normal):", sum(ctrl), "\n")

# coppie complete paziente tumore/normale (servono per l'analisi appaiata)
paired_ids <- intersect(clin$patient[case], clin$patient[ctrl])
cat("Pazienti con entrambi i campioni:", length(paired_ids), "\n")
```

Spiegazione del codice

- `case` e `ctrl` sono vettori logici lunghi 116 usati per selezionare le colonne della matrice nei calcoli successivi.
- `paired_ids` contiene gli ID dei pazienti presenti in entrambi i gruppi (57): servono all'analisi di sensibilità appaiata (sezione 15).

Risultato: Casi (Tumor): **58** – Controlli (Normal): **58**. Pazienti con entrambi i campioni: **57** (paziente 3023 solo tumore, paziente 3035 solo normale).

7. Pre-elaborazione (1): trasformazione logaritmica

7.1 Teoria

Le intensità di espressione dei microarray hanno una distribuzione fortemente **asimmetrica a destra**: la maggior parte dei geni ha valori bassi e pochi geni hanno valori molto elevati. Inoltre la varianza cresce con la media (eteroschedasticità). La trasformazione logaritmica in base 2:

- rende la distribuzione approssimativamente simmetrica e più vicina alla normale, requisito dei test parametrici;
- stabilizza la varianza, rendendo confrontabili geni con livelli medi diversi;
- trasforma i rapporti in differenze: un raddoppio dell'espressione corrisponde a +1 in scala log2, un dimezzamento a −1.

Si somma 1 prima del logaritmo (*pseudocount*) perché log2(0) non è definito; per valori grandi l'effetto del +1 è trascurabile, mentre per valori nulli si ottiene log2(1) = 0.

$$y = log2(x + 1)$$

7.2 Codice

Script R – Trasformazione log2

```
## ---- 4. PRE-ELABORAZIONE (a): TRASFORMAZIONE log2(x + 1) --------------------
expr <- expr[complete.cases(expr), ]          # rimozione sonde con NA
if (min(expr) < 0) {                          # Illumina: possibili valori
  cat("Valori negativi:", sum(expr < 0), "-> posti a 0\n")  # negativi dopo
  expr[expr < 0] <- 0                         # sottrazione del background
}
if (max(expr) > 50) {
  logexpr <- log2(expr + 1)                   # dati in scala lineare
} else {
  # i dati risultano gia in scala log2: una seconda trasformazione
  # comprimerebbe artificialmente la variabilita
  message("Dati gia in scala logaritmica: trasformazione non riapplicata")
  logexpr <- expr
}
print(summary(as.vector(logexpr)))

# controllo qualita: distribuzione dei valori per campione
long <- data.frame(sample = rep(colnames(logexpr), each = nrow(logexpr)),
                   value  = as.vector(logexpr),
                   group  = rep(as.character(grp), each = nrow(logexpr)))
p_box <- ggplot(long, aes(sample, value, fill = group)) +
  geom_boxplot(outlier.size = 0.2, linewidth = 0.2) +
  scale_fill_manual(values = c(Normal = "#2E75B6", Tumor = "#C00000")) +
  labs(title = "Distribuzione dell'espressione log2 per campione",
       x = "Campioni", y = "log2(espressione + 1)") +
  theme_bw() + theme(axis.text.x = element_blank(),
                     axis.ticks.x = element_blank())
save_plot(p_box, "03_boxplot_campioni_log2.png", 12, 5)
```

Spiegazione del codice

- `complete.cases` elimina le sonde con valori mancanti, che renderebbero impossibili i calcoli successivi.
- Gli eventuali **valori negativi** (dovuti alla sottrazione del background Illumina) vengono posti a 0, così che log2(0 + 1) = 0.
- **Controllo di scala**: se il valore massimo supera 50 i dati sono in scala lineare e si applica log2(x + 1), come richiesto dalla traccia. Se invece il massimo è inferiore (valori tipicamente tra 0 e 16), i dati sono già log-trasformati dagli autori: una seconda trasformazione comprimerebbe la variabilità e falserebbe il fold-change, quindi lo script lo segnala e non la riapplica.
- Il boxplot delle distribuzioni per campione è un **controllo di qualità**: dati correttamente normalizzati mostrano mediane e quartili allineati; un campione con distribuzione molto diversa è un potenziale outlier tecnico.

Risultato: il valore massimo della matrice è **16,05** (< 50), quindi lo script ha riconosciuto che i dati sono **già log2-trasformati** e non ha riapplicato la trasformazione. Il Series Matrix di GSE32863 contiene infatti i valori normalizzati e log2-trasformati dagli autori. Applicare di nuovo log2(x + 1) avrebbe compresso i valori in un intervallo ristretto (circa 2,9–4,1) e ridotto artificialmente tutti i fold-change. Il passo richiesto dalla traccia è quindi soddisfatto "a monte", e tutte le analisi successive operano correttamente in scala log2. Nel boxplot le distribuzioni dei 116 campioni hanno mediane e quartili praticamente sovrapposti, segno di una normalizzazione efficace (di tipo quantile) e dell'assenza di campioni anomali dal punto di vista tecnico.

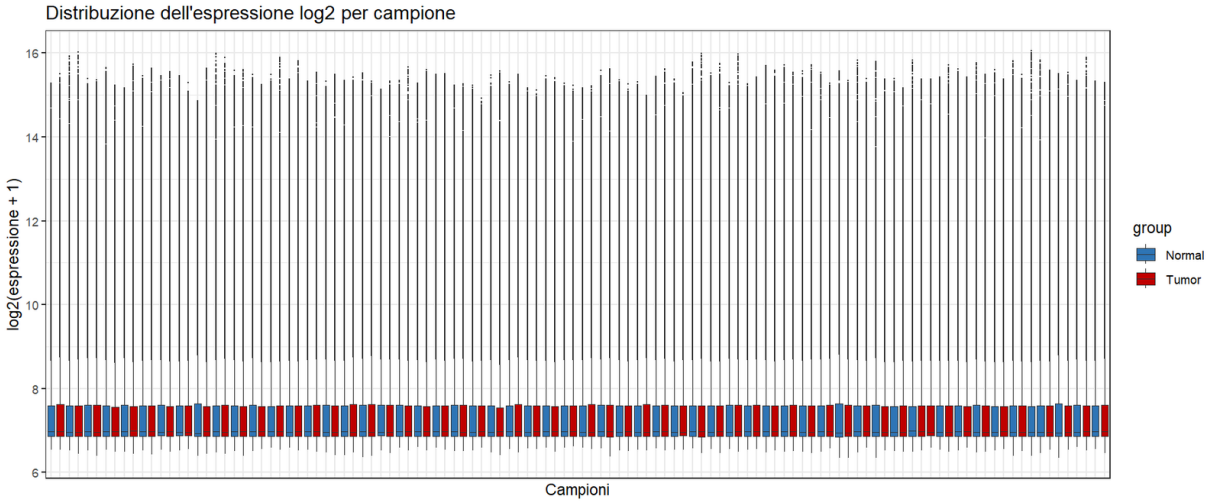


Figura 4 – *Distribuzione dei valori di espressione (scala log2) per ciascuno dei 116 campioni. L'etichetta dell'asse y è quella generica dello script: i valori sono già log2 (file: 03_boxplot_campioni_log2.png)*

8. Pre-elaborazione (2): filtro sull'intervallo interquartile

8.1 Teoria

L'**intervallo interquartile** (IQR) di un gene è la differenza tra il terzo quartile (75° percentile) e il primo quartile (25° percentile) dei suoi valori nei campioni:

$$IQR = Q3 - Q1$$

È una misura di dispersione **robusta**, perché non è influenzata dai valori estremi (a differenza della varianza o del range). Un gene con IQR molto basso ha espressione quasi costante in tutti i campioni: non può discriminare i gruppi e spesso corrisponde a sonde non espresse che misurano solo rumore di fondo.

Il filtro IQR è un **filtro non specifico** (*independent filtering*): non utilizza le etichette Tumor/Normal, quindi non introduce distorsioni nel test successivo. Il suo vantaggio è ridurre il numero di test eseguiti: con meno ipotesi da correggere, la correzione per test multipli diventa meno penalizzante e la potenza statistica aumenta (Bourgon et al., 2010).

La soglia si sceglie osservando l'**istogramma della distribuzione dell'IQR**: tipicamente si osserva un picco di geni con IQR prossimo a zero (geni "piatti") seguito da una coda. La soglia va posta in corrispondenza della fine di questo picco; la traccia suggerisce come punto di partenza il 10° percentile.

8.2 Codice

Script R – Filtro IQR

```
## ---- 5. PRE-ELABORAZIONE (b): FILTRO SULL'INTERVALLO INTERQUARTILE ---------
iqr <- rowIQRs(logexpr)                      # IQR = Q3 - Q1 per ogni sonda
names(iqr) <- rownames(logexpr)
iqr_thr <- quantile(iqr, probs = IQR_PERC)   # soglia al percentile scelto
cat("Soglia IQR (", IQR_PERC * 100, "deg percentile):", round(iqr_thr, 4), "\n")

p_iqr <- ggplot(data.frame(iqr = iqr), aes(iqr)) +
  geom_histogram(bins = 100, fill = "grey60", colour = "grey30") +
  geom_vline(xintercept = iqr_thr, colour = "red", linewidth = 1,
             linetype = "dashed") +
  annotate("text", x = iqr_thr, y = Inf, vjust = 2, hjust = -0.05,
           label = paste0(IQR_PERC * 100, " percentile = ",
                          round(iqr_thr, 3)), colour = "red") +
  labs(title = "Distribuzione dell'IQR delle sonde",
       x = "IQR (scala log2)", y = "Frequenza") + theme_bw()
save_plot(p_iqr, "04_istogramma_IQR.png", 8, 5)

keep <- iqr > iqr_thr
filt <- logexpr[keep, ]
cat("Sonde prima:", nrow(logexpr), " - dopo filtro IQR:", nrow(filt), "\n")
```

Spiegazione del codice

- `rowIQRs` (pacchetto matrixStats) calcola l'IQR di ogni riga in modo vettorializzato, molto più rapidamente di `apply(logexpr, 1, IQR)`.
- `quantile(iqr, probs = IQR_PERC)` restituisce il valore di IQR corrispondente al percentile scelto (10° di default).

- L'istogramma con la linea rossa tratteggiata mostra dove cade la soglia rispetto alla distribuzione: se il picco dei geni poco variabili si estende oltre il 10° percentile, si aumenta `IQR_PERC` (es. 0.20 o 0.25) e si riesegue lo script.
- `keep <- iqr > iqr_thr` è un vettore logico usato per conservare solo le sonde sufficientemente variabili.

Risultato: soglia IQR al 10° percentile = **0,0805**; sonde prima del filtro **48.803**, dopo il filtro **43.922** (eliminate 4.881).

Commento dell'istogramma. La distribuzione dell'IQR è fortemente bimodale: un picco molto alto e stretto di sonde con IQR inferiore a circa 0,2, seguito da una seconda popolazione più ampia di sonde variabili (IQR tra 0,3 e 1,5) e da una lunga coda fino a oltre 4. Il picco corrisponde alle sonde non espresse nel tessuto polmonare, il cui segnale è al livello del fondo (circa 6,5–7 in scala log2: la mediana globale dei dati è infatti 6,97). La soglia al 10° percentile cade **all'interno** di questo picco, quindi elimina solo una parte delle sonde non informative; l'istogramma suggerirebbe una soglia più alta, in corrispondenza della valle tra le due popolazioni (indicativamente attorno al 50° percentile).

Si è comunque mantenuto il 10° percentile, valore proposto dalla traccia, perché la scelta non altera l'elenco finale dei DEG: una sonda con IQR complessivo inferiore a 0,2 non può avere una differenza di 1 unità log2 tra due gruppi di uguale numerosità (se le medie dei gruppi differissero di 1, il primo e il terzo quartile dei 116 valori cadrebbero in gruppi diversi e l'IQR sarebbe prossimo a 1). L'unico effetto è un numero maggiore di test, che rende la correzione FDR leggermente più conservativa. Una soglia più alta resta comunque un'alternativa valida, ottenibile modificando il parametro `IQR_PERC`.

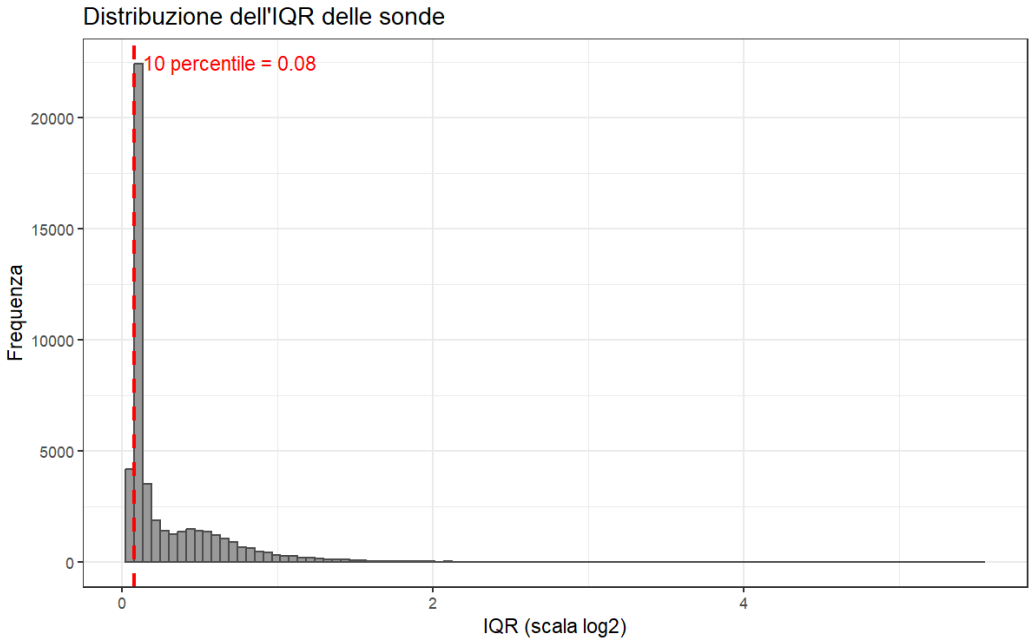


***Figura 5** – Istogramma della distribuzione dell'IQR delle sonde con la soglia al 10° percentile (linea rossa) (file: 04_istogramma_IQR.png)*

9. Annotazione: da sonda a gene

Le righe della matrice sono identificativi di sonda Illumina (es. `ILMN_1343291`). Per l'interpretazione biologica e per Enrichr servono i **simboli genici ufficiali HGNC**. La tabella della piattaforma GPL6884 contiene la colonna `Symbol`.

Quando più sonde mappano lo stesso gene, si conserva **la sonda con IQR maggiore**, cioè quella più informativa. È una strategia comune (alternativa: media delle sonde o sonda con espressione media più alta); il vantaggio è evitare che lo stesso gene compaia più volte tra i DEG e nelle liste per l'arricchimento.

Script R – Annotazione e collasso delle sonde

```
## ---- 6. ANNOTAZIONE: DA SONDA A SIMBOLO GENICO ---------------------------
# Piu sonde possono mappare lo stesso gene: si conserva quella con IQR
# maggiore (la piu informativa); si scartano le sonde senza simbolo.
sym_col <- intersect(c("Symbol", "ILMN_Gene", "Gene symbol", "GENE_SYMBOL"),
                     colnames(fd))[1]
ann <- data.frame(probe  = rownames(filt),
                  symbol = as.character(fd[rownames(filt), sym_col]),
                  iqr    = iqr[rownames(filt)],
                  stringsAsFactors = FALSE)
ann <- ann[!is.na(ann$symbol) & ann$symbol != "", ]
ann <- ann[order(ann$symbol, -ann$iqr), ]
ann <- ann[!duplicated(ann$symbol), ]
gexpr <- filt[ann$probe, ]
rownames(gexpr) <- ann$symbol
cat("Geni unici dopo il collasso delle sonde:", nrow(gexpr), "\n")
```

Spiegazione del codice

- `sym_col` individua il nome della colonna con il simbolo genico tra alcune alternative comuni, rendendo il codice robusto a variazioni nella tabella GPL.
- Le sonde senza simbolo (sonde sperimentali o non annotate) vengono eliminate.
- Ordinando per simbolo e IQR decrescente, `!duplicated(ann$symbol)` mantiene per ogni gene solo la prima occorrenza, cioè la sonda a IQR massimo.

Risultato: dopo l'eliminazione delle sonde senza simbolo e il collasso delle sonde multiple si ottengono **23.949 geni unici**, che costituiscono l'universo dei geni testati.

10. Filtraggio statistico: fold-change, test t e FDR

10.1 Il fold-change in scala logaritmica

Il **fold-change** (FC) misura di quante volte cambia l'espressione media nei casi rispetto ai controlli. Poiché i dati sono in scala log2, il logaritmo del rapporto diventa la differenza delle medie:

$$logFC = log2(\ media_caso\ /\ media_controllo\ ) = mean(log2\ caso) - mean(log2\ controllo)$$

(a rigore, la differenza delle medie dei logaritmi corrisponde al log del rapporto delle medie geometriche). Un logFC = 1 indica un raddoppio nel tumore, logFC = −1 un dimezzamento, logFC = 2 un aumento di 4 volte.

10.2 Scelta del test statistico

Per ogni gene si verifica il sistema di ipotesi:

$$H0: \mu_Tumor = \mu_Normal \qquad H1: \mu_Tumor \neq \mu_Normal$$

Con n = 58 per gruppo (n > 30), per il **teorema del limite centrale** la distribuzione delle medie campionarie è approssimativamente normale indipendentemente dalla distribuzione dei dati originali; si può quindi applicare un **test t di Student a due campioni**. Si è scelta la variante di **Welch**, che non assume varianze uguali nei due gruppi: è un'ipotesi prudente perché il tessuto tumorale è biologicamente più eterogeneo del normale e ha spesso varianza maggiore. La statistica è:

$$t = (\ \bar{x}_T - \bar{x}_N\ ) / \sqrt{(\ s^2_T / n_T + s^2_N / n_N\ )}$$

con gradi di libertà stimati con l'approssimazione di Welch–Satterthwaite. Con gruppi bilanciati e varianze simili il test di Welch e il test di Student classico (`var.equal = TRUE`) danno risultati quasi identici; quando le varianze differiscono, il test di Welch controlla meglio l'errore di primo tipo.

Alternative valide sono il test non parametrico di **Wilcoxon–Mann–Whitney** (se l'ipotesi di normalità è dubbia e il campione è piccolo) e il **t moderato** del pacchetto `limma`, che stabilizza la varianza di ciascun gene "prendendo in prestito" informazione dagli altri geni (approccio Bayesiano empirico). Poiché i campioni sono appaiati, in sezione 15 si esegue anche un test t appaiato come analisi di sensibilità.

10.3 Il problema dei test multipli e il False Discovery Rate

Con circa 20.000 geni testati al livello α = 0,05, anche se nessun gene fosse realmente differenziale ci aspetteremmo circa 1.000 falsi positivi per puro caso. È quindi necessario correggere i p-value. La correzione di **Bonferroni** controlla la probabilità di avere anche un solo falso positivo (FWER) ed è troppo conservativa in genomica. Si preferisce controllare il **False Discovery Rate (FDR)**, cioè la proporzione attesa di falsi positivi tra i geni dichiarati significativi:

$$FDR = E[\ V / R\ ] \quad (V = falsi\ positivi,\ R = geni\ dichiarati\ significativi)$$

La procedura di **Benjamini–Hochberg** (1995) ordina gli m p-value in ordine crescente p(1) ≤ … ≤ p(m) e calcola il p-value aggiustato:

$$p_adj(i) = min_\{k \geq i\}\ \{\ min(\ m \cdot p(k) / k\ ,\ 1\ )\ \}$$

Selezionare i geni con p_adj < 0,05 significa accettare che, in media, al massimo il 5% di essi sia un falso positivo.

10.4 Codice

Script R – logFC, test t e FDR

```
## ---- 7. FILTRAGGIO: logFC, TEST t, CORREZIONE FDR -------------------------
# logFC: i dati sono in scala log2, quindi il rapporto diventa differenza
logFC <- rowMeans(gexpr[, case]) - rowMeans(gexpr[, ctrl])

# n = 58 per gruppo (> 30): per il teorema del limite centrale la media
# campionaria e approssimativamente normale -> test t a due campioni.
# Si usa la variante di Welch (varianze non assunte uguali).
pval <- apply(gexpr, 1, function(x)
  t.test(x[case], x[ctrl], var.equal = FALSE)$p.value)

# correzione per test multipli: False Discovery Rate di Benjamini-Hochberg
padj <- p.adjust(pval, method = "BH")

res <- data.frame(Gene = rownames(gexpr), logFC = logFC,
                  AveExpr = rowMeans(gexpr), P.Value = pval,
                  adj.P.Val = padj, stringsAsFactors = FALSE)
res <- res[order(res$adj.P.Val), ]
print(head(res, 10))
cat("Geni con FDR <", ALPHA, ":", sum(res$adj.P.Val < ALPHA), "\n")

# istogramma dei p-value grezzi (diagnostica: picco vicino a 0 = segnale)
p_ph <- ggplot(res, aes(P.Value)) +
  geom_histogram(bins = 50, fill = "#7F7F7F", colour = "white") +
  labs(title = "Distribuzione dei p-value (test t di Welch)",
       x = "p-value", y = "Frequenza") + theme_bw()
save_plot(p_ph, "05_istogramma_pvalue.png", 7, 4.5)
```

Spiegazione del codice

- `rowMeans` calcola le medie per riga nei due gruppi; la differenza è il logFC per ciascun gene.
- `apply(gexpr, 1, …)` esegue un `t.test` per ogni gene: `var.equal = FALSE` corrisponde al test di Welch (default di R) e si estrae il campo `$p.value`.
- `p.adjust(pval, method = "BH")` applica la correzione di Benjamini–Hochberg.
- I risultati sono raccolti nel data frame `res`, ordinato per p-value aggiustato. `AveExpr` è l'espressione media del gene, utile per interpretare i risultati (geni poco espressi hanno misure meno affidabili).
- L'**istogramma dei p-value** è una diagnostica: in presenza di segnale reale si osserva un picco vicino a 0 sopra una distribuzione uniforme (attesa per i geni non differenziali).

Risultato: i geni con FDR < 0,05 sono **11.495**, circa il 48% dei 23.949 geni testati. Il numero elevato è atteso: con 58 campioni per gruppo il test ha una potenza molto alta e rileva come significative anche variazioni piccole. Per questo motivo è indispensabile la soglia aggiuntiva sul fold-change (sezione 11). I 10 geni più significativi sono tutti down-regolati nel tumore:

Gene	logFC	AveExpr	P.Value	adj.P.Val
ADH1B	−2,67	8,61	$9{,}96 \times 10^{-57}$	$2{,}39 \times 10^{-52}$
FMO2	−3,12	10,05	$5{,}12 \times 10^{-53}$	$6{,}13 \times 10^{-49}$
AGER	−4,31	10,13	$8{,}54 \times 10^{-53}$	$6{,}82 \times 10^{-49}$

Gene	logFC	AveExpr	P.Value	adj.P.Val
CLEC3B	−3,67	9,93	$1{,}48 \times 10^{-52}$	$8{,}89 \times 10^{-49}$
FAM107A	−3,87	9,84	$1{,}05 \times 10^{-51}$	$5{,}04 \times 10^{-48}$
ITLN2	−3,92	9,18	$3{,}88 \times 10^{-48}$	$1{,}55 \times 10^{-44}$
STX11	−2,36	8,97	$5{,}58 \times 10^{-48}$	$1{,}91 \times 10^{-44}$
TCF21	−3,21	9,21	$8{,}09 \times 10^{-47}$	$2{,}42 \times 10^{-43}$
JAM2	−1,88	8,55	$9{,}28 \times 10^{-47}$	$2{,}47 \times 10^{-43}$
PLAC9	−3,07	9,91	$2{,}33 \times 10^{-46}$	$5{,}07 \times 10^{-43}$

Tabella – I 10 geni con p-value aggiustato più basso (output di head(res, 10)).

L'istogramma dei p-value mostra un picco molto pronunciato vicino a 0 (circa 10.500 geni nel primo intervallo) sopra una distribuzione approssimativamente uniforme per i valori più alti: è il profilo atteso quando una frazione ampia dei geni è realmente differenziale, e conferma che il test è ben calibrato.

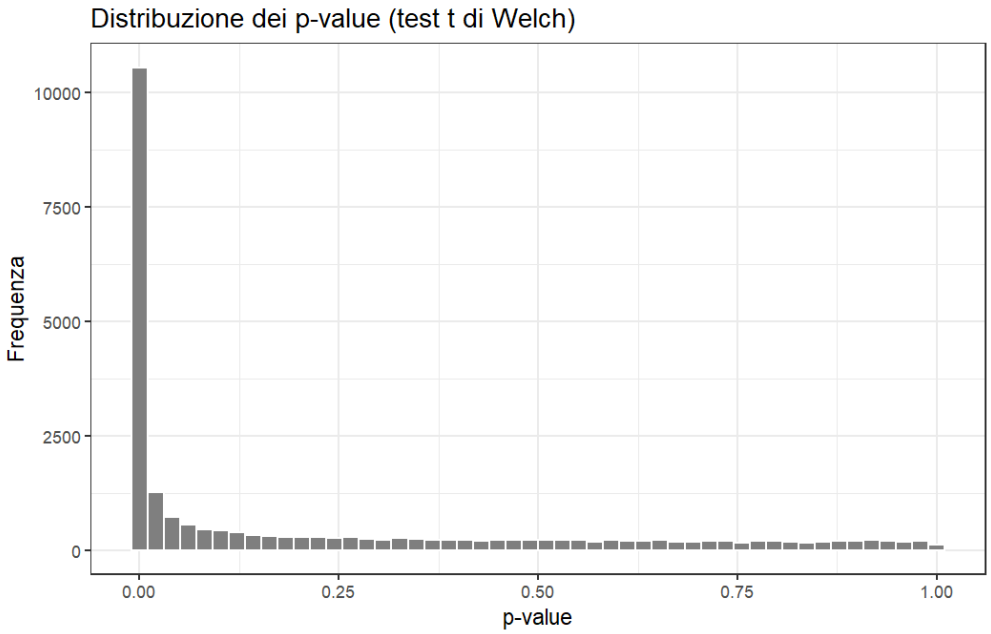


Figura 6 – *Distribuzione dei p-value grezzi del test t di Welch (file: 05_istogramma_pvalue.png)*

11. Volcano plot e selezione dei DEG

11.1 Teoria

Il **volcano plot** riassume in un'unica figura le due dimensioni della differenza di espressione: sull'asse x il **logFC** (entità dell'effetto biologico) e sull'asse y il **−log10(p-value aggiustato)** (evidenza statistica). I geni più interessanti si trovano negli angoli superiori: grande variazione e alta significatività.

Si combinano due soglie perché misurano aspetti diversi. Con campioni numerosi anche variazioni minime possono risultare statisticamente significative ma biologicamente irrilevanti; la soglia sul fold-change elimina questi geni. Si è scelto **FC = 2**, quindi |logFC| ≥ log2(2) = 1 (raddoppio o dimezzamento), e **FDR < 0,05**:

- **Up-regolato**: logFC ≥ 1 e p_adj < 0,05 (più espresso nel tumore).
- **Down-regolato**: logFC ≤ −1 e p_adj < 0,05 (meno espresso nel tumore).
- **Non significativo (NS)**: tutti gli altri geni.

11.2 Codice

Script R – Volcano plot e soglie

```
## ---- 8. VOLCANO PLOT E APPLICAZIONE DELLE SOGLIE --------------------------
lfc_thr <- log2(FC_THR)      # soglia sul logFC (1 se FC_THR = 2)
res$Regulation <- "NS"
res$Regulation[res$logFC >=  lfc_thr & res$adj.P.Val < ALPHA] <- "Up"
res$Regulation[res$logFC <= -lfc_thr & res$adj.P.Val < ALPHA] <- "Down"
print(table(res$Regulation))

top_lab <- rbind(head(res[res$Regulation == "Up", ][
                   order(-res$logFC[res$Regulation == "Up"]), ], 10),
                 head(res[res$Regulation == "Down", ][
                   order(res$logFC[res$Regulation == "Down"]), ], 10))

p_vol <- ggplot(res, aes(logFC, -log10(adj.P.Val), colour = Regulation)) +
  geom_point(size = 0.8, alpha = 0.6) +
  scale_colour_manual(values = c(Down = "#2E75B6", NS = "grey70",
                                 Up = "#C00000")) +
  geom_vline(xintercept = c(-lfc_thr, lfc_thr), linetype = "dashed") +
  geom_hline(yintercept = -log10(ALPHA), linetype = "dashed") +
  geom_text_repel(data = top_lab, aes(label = Gene), size = 3,
                  max.overlaps = 30, show.legend = FALSE) +
  labs(title = "Volcano plot - LUAD vs tessuto normale",
       x = "log2 Fold Change (Tumor - Normal)",
       y = "-log10(p-value aggiustato FDR)") + theme_bw()
save_plot(p_vol, "06_volcano_plot.png", 9, 7)

# sottoinsieme finale dei geni differenzialmente espressi (DEG)
deg <- res[res$Regulation != "NS", ]
deg <- deg[order(-abs(deg$logFC)), ]
cat("DEG totali:", nrow(deg), " - Up:", sum(deg$Regulation == "Up"),
    " - Down:", sum(deg$Regulation == "Down"), "\n")
```

Spiegazione del codice

- `lfc_thr <- log2(FC_THR)` converte la soglia di fold-change lineare nella scala log2.
- La colonna `Regulation` classifica ogni gene come Up, Down o NS in base alle due soglie congiunte.

- Il grafico colora i geni per categoria; le **linee tratteggiate** indicano le soglie (verticali a ±1, orizzontale a −log10(0,05) ≈ 1,3). `geom_text_repel` etichetta i 10 geni più up- e i 10 più down-regolati evitando sovrapposizioni.
- `deg` contiene solo i geni significativi, ordinati per |logFC| decrescente.

Risultato: applicando |logFC| ≥ 1 e FDR < 0,05 si ottengono **1.364 DEG**: **580 up-regolati** e **784 down-regolati**; 22.585 geni sono non significativi.

Il volcano plot ha la tipica forma a "V" ed è **asimmetrico**: i geni down-regolati raggiungono fold-change più estremi (fino a logFC ≈ −4,8) e significatività maggiore (−log10 FDR fino a circa 51) rispetto agli up-regolati (logFC massimo ≈ 3,4). Questa asimmetria è caratteristica del confronto tumore/polmone normale: il tessuto normale esprime ad alti livelli geni specifici di cellule che nel tumore sono quasi assenti (pneumociti, endotelio capillare, macrofagi alveolari, sangue), mentre l'aumento dei geni tumorali è più eterogeneo tra pazienti. Tra i geni etichettati compaiono, per i down-regolati, AGER, ITLN2, MCEMP1, CA4, HBB, HBA2, SFTPA1 e CLDN18 e, per gli up-regolati, SPP1, COL1A1, MMP11, GCNT3, CRABP2, CEACAM5, SPINK1, EEF1A2 e XAGE1.

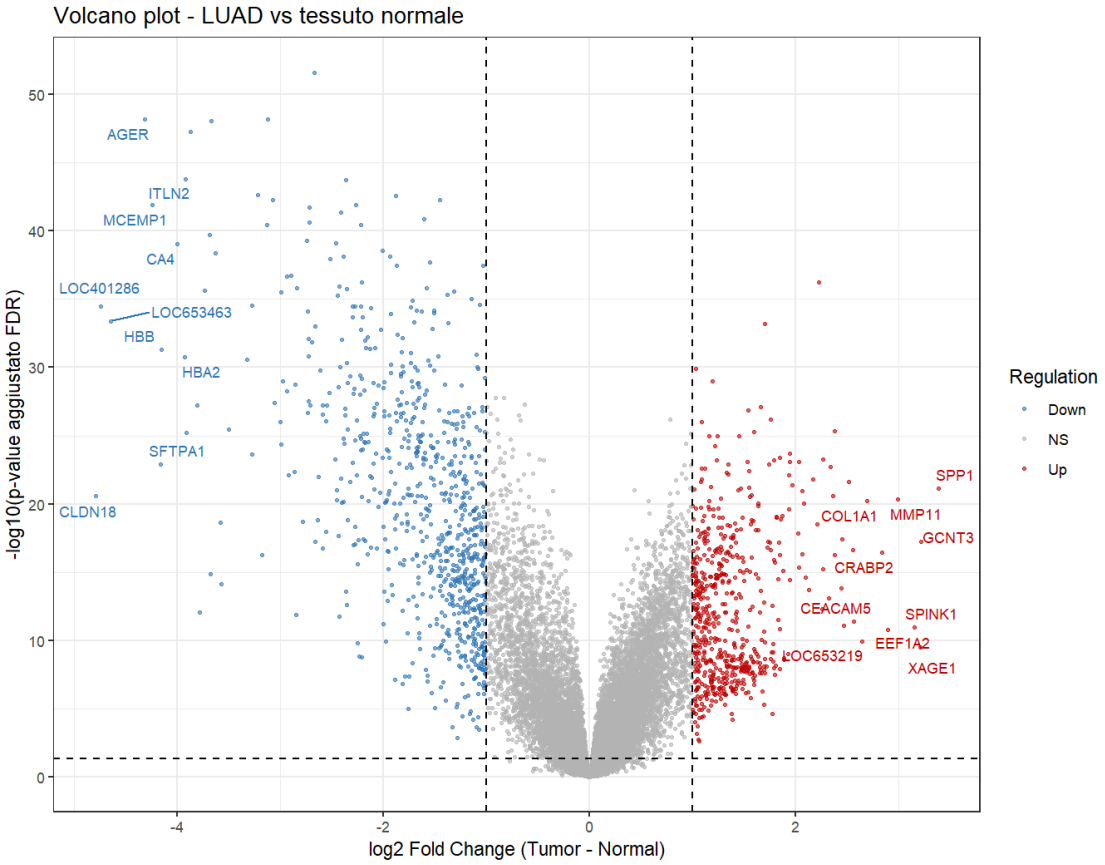


Figura 7 – *Volcano plot Tumor vs Normal. Rosso: up-regolati; blu: down-regolati; linee tratteggiate: soglie |logFC| = 1 e FDR = 0,05 (file: 06_volcano_plot.png)*

12. Grafici e ulteriori analisi

12.1 Boxplot dei geni più up- e down-regolati

Il **boxplot** rappresenta la distribuzione di un gene in ciascun gruppo: la linea centrale è la mediana, la scatola va dal primo al terzo quartile (la sua altezza è l'IQR), i baffi si estendono fino a 1,5 × IQR. Sovrapponendo i singoli campioni (jitter) si valuta visivamente quanto i due gruppi siano separati e se la differenza sia guidata da pochi outlier.

Script R – Boxplot

```
## ---- 9. BOXPLOT DEL GENE PIU UP-REGOLATO E PIU DOWN-REGOLATO --------------
g_up   <- deg$Gene[which.max(deg$logFC)]
g_down <- deg$Gene[which.min(deg$logFC)]
cat("Gene piu up-regolato:", g_up, " - piu down-regolato:", g_down, "\n")

bx <- data.frame(Expression = c(gexpr[g_up, ], gexpr[g_down, ]),
                 Gene  = rep(c(paste(g_up, "(UP)"), paste(g_down, "(DOWN)")),
                             each = ncol(gexpr)),
                 Group = rep(as.character(grp), 2))
p_bx <- ggplot(bx, aes(Group, Expression, fill = Group)) +
  geom_boxplot(outlier.shape = NA, alpha = 0.8) +
  geom_jitter(width = 0.15, size = 0.8, alpha = 0.6) +
  facet_wrap(~ Gene, scales = "free_y") +
  scale_fill_manual(values = c(Normal = "#2E75B6", Tumor = "#C00000")) +
  labs(y = "log2(espressione + 1)", x = NULL,
       title = "Geni con la massima variazione") +
  theme_bw() + theme(legend.position = "none")
save_plot(p_bx, "07_boxplot_top_up_down.png", 9, 5)
```

Spiegazione del codice

- `which.max(deg$logFC)` e `which.min(deg$logFC)` identificano il gene più up-regolato e quello più down-regolato.
- I valori dei due geni sono riuniti in un data frame in formato "long" e rappresentati con `facet_wrap`, un pannello per gene con scala y indipendente.

Risultato: gene più up-regolato: **SPP1** (osteopontina; logFC = +3,39, FC ≈ 10,5; FDR = 7,9 × 10^{-22}). Gene più down-regolato: **CLDN18** (claudina-18; logFC = −4,79, FC ≈ 1/28; FDR = 2,8 × 10^{-21}).

Nei controlli CLDN18 ha un'espressione molto alta e omogenea (circa 14 in scala log2), mentre nei tumori la distribuzione è ampia: la maggior parte dei campioni scende a 7–10, ma alcuni tumori conservano livelli vicini al normale. Questi casi possono corrispondere a tumori ben differenziati, che mantengono caratteristiche alveolari, o a campioni con una quota maggiore di parenchima normale. SPP1 mostra il comportamento opposto: bassa espressione nei normali (circa 7–9,5) e netto aumento nella quasi totalità dei tumori (mediana circa 11,5).

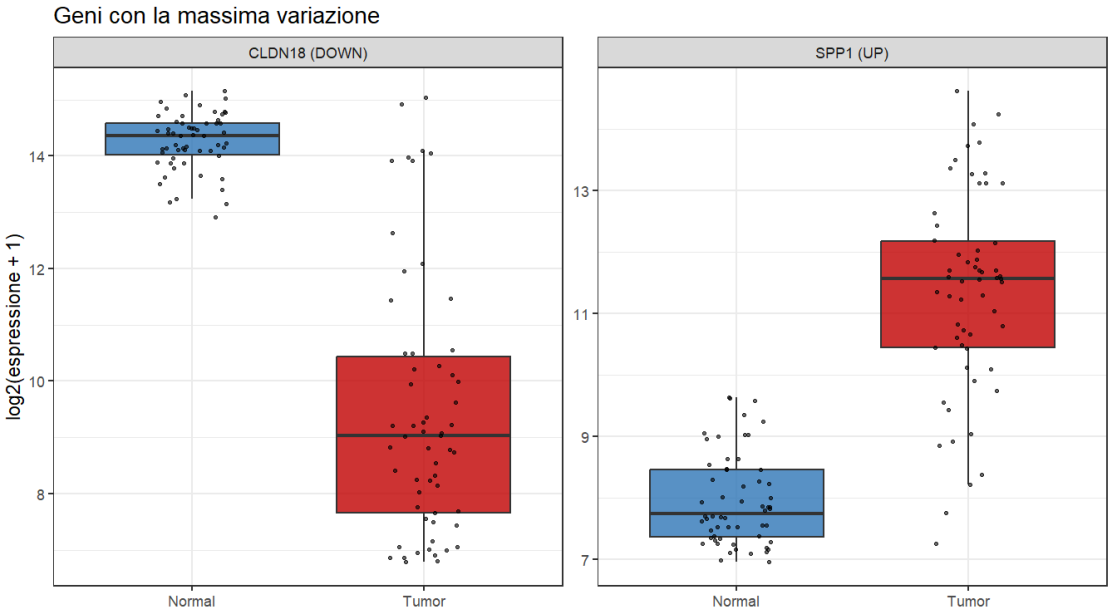


***Figura 8** – Boxplot del gene più down-regolato (CLDN18) e del più up-regolato (SPP1) (file: 07_boxplot_top_up_down.png)*

12.2 Clustering gerarchico e heatmap

Il **clustering gerarchico agglomerativo** parte da elementi singoli e li unisce iterativamente in gruppi sempre più grandi, producendo un albero (**dendrogramma**). Richiede due scelte:

- **Misura di distanza**: si usa la distanza di correlazione d = 1 − r (r = correlazione di Pearson), che raggruppa geni o campioni con lo stesso andamento indipendentemente dal livello assoluto.
- **Criterio di collegamento** (*linkage*): l'*average linkage* definisce la distanza tra due cluster come la media delle distanze tra tutte le coppie di elementi; è un compromesso tra *single* (cluster allungati) e *complete* (cluster compatti).

Nella **heatmap** ogni cella rappresenta l'espressione di un gene (riga) in un campione (colonna). I valori sono standardizzati per riga (**z-score**: z = (x − media)/deviazione standard), così che il colore indichi se il gene è sopra (rosso) o sotto (blu) la propria media, rendendo confrontabili geni con livelli di espressione diversi. Se i DEG sono informativi, il clustering dei campioni dovrebbe separare spontaneamente tumori e normali, pur non avendo usato le etichette.

Script R – Clustering e heatmap

```
## ---- 10. CLUSTERING E HEATMAP DEI DEG -----------------------------------
deg_mat <- gexpr[deg$Gene, ]
ann_col <- data.frame(Group = grp, row.names = colnames(deg_mat))
ann_colors <- list(Group = c(Normal = "#2E75B6", Tumor = "#C00000"))

# a) tutti i DEG: z-score per riga, distanza di correlazione, legame average
pheatmap(deg_mat, scale = "row",
         clustering_distance_rows = "correlation",
         clustering_distance_cols = "correlation",
         clustering_method = "average",
         color = colorRampPalette(rev(brewer.pal(11, "RdBu")))(100),
         breaks = seq(-3, 3, length.out = 101),
         annotation_col = ann_col, annotation_colors = ann_colors,
         show_rownames = nrow(deg_mat) <= 60, show_colnames = FALSE,
         main = paste("Heatmap dei", nrow(deg_mat), "DEG"),
         filename = file.path(OUT_DIR, "08_heatmap_DEG.png"),
         width = 10, height = 10)
```

```
# b) i 50 DEG con |logFC| maggiore, con i nomi dei geni leggibili
top50 <- head(deg$Gene, 50)
pheatmap(gexpr[top50, ], scale = "row",
         clustering_distance_rows = "correlation",
         clustering_distance_cols = "correlation",
         clustering_method = "average",
         color = colorRampPalette(rev(brewer.pal(11, "RdBu")))(100),
         breaks = seq(-3, 3, length.out = 101),
         annotation_col = ann_col, annotation_colors = ann_colors,
         show_colnames = FALSE, fontsize_row = 7,
         main = "Top 50 DEG per |logFC|",
         filename = file.path(OUT_DIR, "09_heatmap_top50.png"),
         width = 10, height = 10)

# raggruppamento dei campioni in 2 cluster e confronto con i gruppi reali
hc <- hclust(as.dist(1 - cor(deg_mat)), method = "average")
cl <- cutree(hc, k = 2)
print(table(Cluster = cl, Gruppo = grp))
```

Spiegazione del codice

- `scale = "row"` applica lo z-score per gene; `breaks` limita la scala colori a ±3 deviazioni standard per non far dominare i valori estremi.
- `annotation_col` aggiunge sopra la heatmap una barra che indica il gruppo di ciascun campione, per verificare visivamente la coerenza tra cluster e gruppi.
- La prima heatmap include tutti i DEG (nomi dei geni nascosti se sono più di 60); la seconda i 50 DEG con |logFC| maggiore, con i nomi leggibili.
- `hclust` + `cutree(k = 2)` tagliano il dendrogramma dei campioni in due cluster; la tabella di contingenza cluster × gruppo quantifica la separazione.

> **Risultato:** il taglio del dendrogramma dei campioni in due cluster riproduce **perfettamente** i gruppi: cluster 1 = 58 Normal e 0 Tumor; cluster 2 = 0 Normal e 58 Tumor.

Nella heatmap di tutti i DEG i campioni si separano nei due rami principali del dendrogramma, e i geni si organizzano in due grandi blocchi con andamento speculare (alti nei normali e bassi nei tumori, o viceversa). All'interno dei normali si nota un sottogruppo con un profilo intermedio, e all'interno dei tumori la colorazione è più eterogenea, a riflettere la diversità biologica dei tumori. La heatmap dei 50 DEG con |logFC| maggiore contiene 43 geni down-regolati e solo 7 up-regolati (SPINK1, GCNT3, SPP1, MMP11, EEF1A2, XAGE1, CRABP2), coerentemente con l'asimmetria del volcano plot. I geni down-regolati formano moduli riconoscibili: surfattante e epitelio alveolare (SFTPC, SFTPA1, SFTPA2, AGER, CLDN18), endotelio e caveole (CA4, CAV1, CAV2), emoglobine (HBA1, HBA2, HBB), macrofagi e immunità innata (MARCO, FCN3, MCEMP1, S100A8), adipociti e fibroblasti (FABP4, CFD, TCF21).

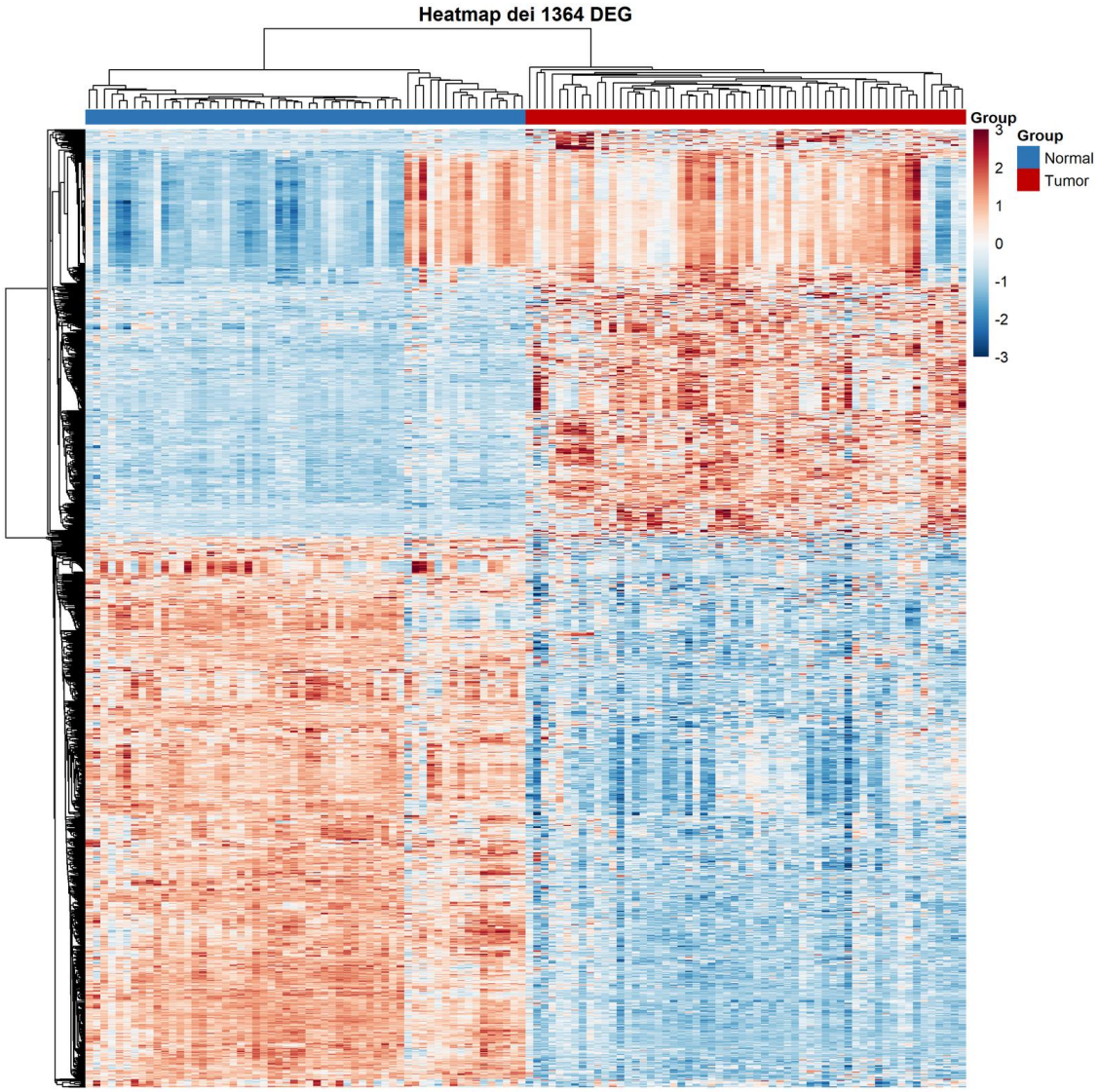


Figura 9 – *Heatmap dei 1.364 DEG (z-score per riga, distanza di correlazione, average linkage) (file: 08_heatmap_DEG.png)*

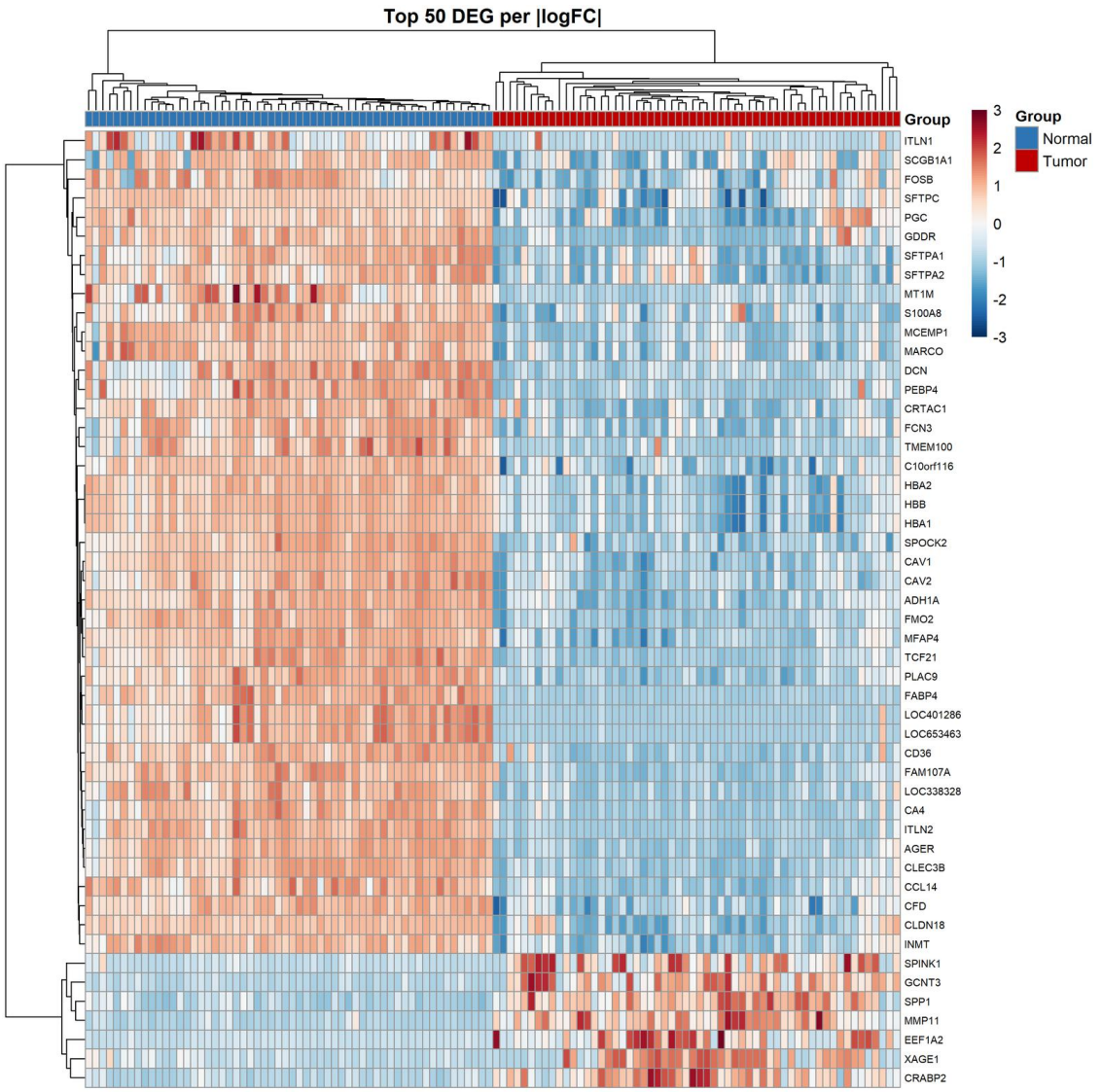


***Figura 10** – Heatmap dei 50 DEG con |logFC| maggiore (file: 09_heatmap_top50.png)*

12.3 Analisi delle componenti principali (PCA)

La **PCA** è una tecnica di riduzione della dimensionalità che trasforma le variabili originali (geni) in nuove variabili non correlate, le **componenti principali**, ordinate per varianza spiegata decrescente. Matematicamente le componenti sono gli autovettori della matrice di covarianza (o di correlazione, se i dati sono standardizzati) e la varianza spiegata da ciascuna è proporzionale all'autovalore corrispondente. Proiettando i campioni sulle prime due componenti si ottiene una rappresentazione 2D che conserva la maggior parte della variabilità.

Script R – PCA

```
## ---- 11. (OPZIONALE) PCA SUI DEG ----------------------------------------
pca <- prcomp(t(deg_mat), center = TRUE, scale. = TRUE)
var_exp <- 100 * pca$sdev^2 / sum(pca$sdev^2)
pc_df <- data.frame(PC1 = pca$x[, 1], PC2 = pca$x[, 2], Group = grp)
p_pca <- ggplot(pc_df, aes(PC1, PC2, colour = Group)) +
  geom_point(size = 2, alpha = 0.8) +
  stat_ellipse(level = 0.95, linetype = "dashed") +
  scale_colour_manual(values = c(Normal = "#2E75B6", Tumor = "#C00000")) +
  labs(title = "PCA dei campioni sui DEG",
       x = sprintf("PC1 (%.1f%%)", var_exp[1]),
       y = sprintf("PC2 (%.1f%%)", var_exp[2])) + theme_bw()
```

```
save_plot(p_pca, "10_PCA_DEG.png", 7, 5.5)

p_scree <- ggplot(data.frame(PC = 1:10, Var = var_exp[1:10]), aes(PC, Var)) +
  geom_col(fill = "#2E75B6") + geom_line() + geom_point() +
  scale_x_continuous(breaks = 1:10) +
  labs(title = "Scree plot", x = "Componente principale",
       y = "% varianza spiegata") + theme_bw()
save_plot(p_scree, "11_scree_plot.png", 6, 4)
```

Spiegazione del codice

- `prcomp` richiede i campioni sulle righe, quindi si traspone la matrice con `t()`; `center = TRUE, scale. = TRUE` standardizza ogni gene (PCA sulla matrice di correlazione).
- La percentuale di varianza spiegata è $\text{sdev}^2/\Sigma\ \text{sdev}^2$. Lo **scree plot** mostra quanta varianza cattura ciascuna componente.
- `stat_ellipse` disegna le ellissi di confidenza al 95% per gruppo.

Risultato: PC1 spiega il **53,6%** della varianza e PC2 il **9,1%**; dalla terza componente in poi la varianza spiegata scende sotto il 4% (scree plot con un "gomito" netto dopo PC2).

Lungo PC1 i due gruppi sono completamente separati (normali a valori negativi, tumori a valori positivi), con ellissi di confidenza che non si sovrappongono: la differenza tumore/normale è la principale fonte di variabilità nello spazio dei DEG. I normali sono distribuiti lungo una diagonale che li avvicina progressivamente ai tumori, e un campione normale si trova in posizione intermedia (PC1 ≈ 0): può trattarsi di tessuto adiacente con alterazioni precoci o con una quota di cellule tumorali. La dispersione lungo PC2 riflette l'eterogeneità interna a ciascun gruppo.

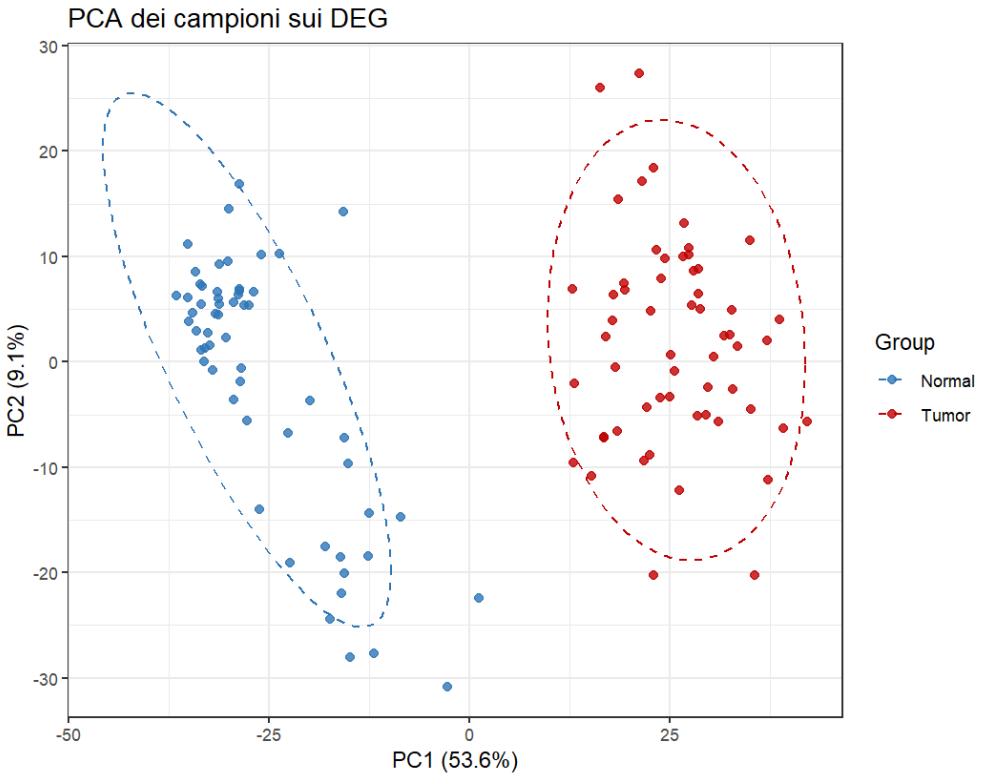


Figura 11 – *PCA dei 116 campioni calcolata sui 1.364 DEG, con ellissi di confidenza al 95% (file: 10_PCA_DEG.png)*

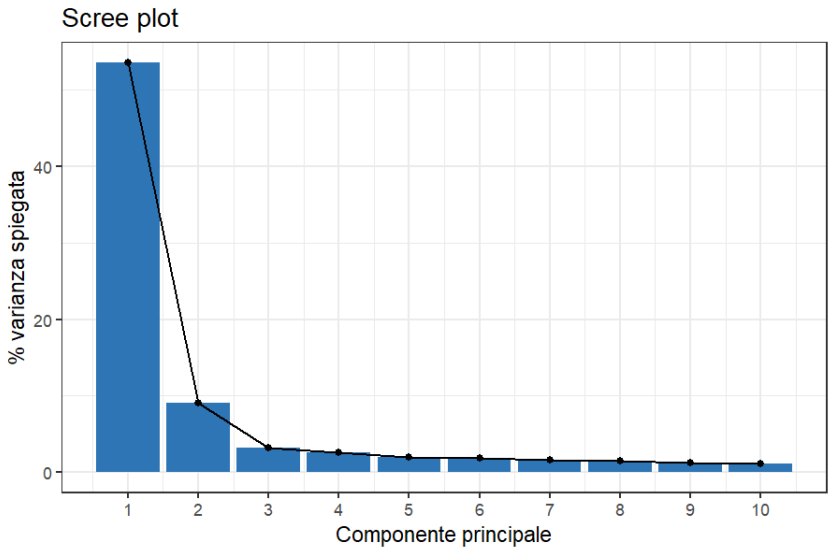


Figura 12 – *Scree plot delle prime 10 componenti principali (file: 11_scree_plot.png)*

13. Esportazione dei risultati

Come richiesto, i risultati vengono salvati in un file di testo tab-delimitato che riporta per ogni DEG: nome del gene, p-value aggiustato, logFC e indicazione UP/DOWN in base al segno del logFC.

Script R – Esportazione

```
## ---- 12. ESPORTAZIONE DEI RISULTATI ---------------------------------------
out <- data.frame(Gene       = deg$Gene,
                  adj.P.Val  = signif(deg$adj.P.Val, 4),
                  logFC      = round(deg$logFC, 4),
                  Regulation = ifelse(deg$logFC > 0, "UP", "DOWN"))
write.table(out, file.path(OUT_DIR, "DEG_GSE32863.txt"),
            sep = "\t", quote = FALSE, row.names = FALSE)
# tabella completa (tutti i geni testati) e liste per EnrichR web
write.table(res, file.path(OUT_DIR, "risultati_completi_GSE32863.txt"),
            sep = "\t", quote = FALSE, row.names = FALSE)
writeLines(out$Gene[out$Regulation == "UP"],
           file.path(OUT_DIR, "lista_UP.txt"))
writeLines(out$Gene[out$Regulation == "DOWN"],
           file.path(OUT_DIR, "lista_DOWN.txt"))
```

Spiegazione del codice

- `DEG_GSE32863.txt` è il file richiesto dalla traccia (colonne: Gene, adj.P.Val, logFC, Regulation).
- `risultati_completi_GSE32863.txt` contiene tutti i geni testati con p-value grezzi e aggiustati, utile per verifiche o per analisi di tipo GSEA.
- `lista_UP.txt` e `lista_DOWN.txt` contengono un gene per riga e possono essere incollati direttamente nell'interfaccia web di Enrichr.

Esempio del formato del file `DEG_GSE32863.txt`:

Formato del file di output

```
Gene    adj.P.Val       logFC   Regulation
<GENE_1><p_adj> <logFC> UP
<GENE_2><p_adj> <logFC> DOWN
...
```

> **Risultato:** il file `DEG_GSE32863.txt` contiene **1.364 righe** (580 UP e 784 DOWN), ordinate per |logFC| decrescente. Le prime 15 righe sono:

output/DEG_GSE32863.txt – prime 15 righe

```
Gene    adj.P.Val       logFC   Regulation
CLDN18  2.765e-21       -4.7909 DOWN
LOC401286       3.507e-35       -4.7371 DOWN
LOC653463       3.958e-34       -4.64   DOWN
AGER    6.816e-49       -4.3094 DOWN
```

```
MCEMP1   1.239e-42       -4.238  DOWN
SFTPA1   1.219e-23       -4.1602 DOWN
HBB      4.877e-32       -4.1489 DOWN
CA4      9.41e-40-3.9992 DOWN
HBA2     1.703e-31       -3.9249 DOWN
ITLN2    1.549e-44       -3.9156 DOWN
FCN3     6.276e-26       -3.9114 DOWN
FAM107A 5.037e-48       -3.8653 DOWN
HBA1     6.177e-28       -3.8067 DOWN
SCGB1A1 8.484e-13       -3.7763 DOWN
ADH1A    2.245e-36       -3.73   DOWN
```

I 10 geni più up-regolati sono: SPP1 (logFC 3,39), GCNT3 (3,22), XAGE1 (3,21), SPINK1 (3,16), MMP11 (3,00), EEF1A2 (2,90), CRABP2 (2,84), COL1A1 (2,69), LOC653219 (2,65) e CEACAM5 (2,56). Tra i DEG compaiono 57 identificativi LOC…: sono sonde associate a trascritti non caratterizzati nella versione dell'annotazione GPL6884, e vanno interpretati con cautela.

14. Analisi di arricchimento funzionale con Enrichr

14.1 Teoria

Una lista di centinaia di geni è difficile da interpretare singolarmente. L'**analisi di arricchimento** (*Over-Representation Analysis*, ORA) verifica se un insieme di geni funzionalmente correlati (un *gene set*: un termine GO, un pathway KEGG…) sia rappresentato nella lista dei DEG più di quanto atteso per caso.

Il test classico è il **test esatto di Fisher**, equivalente al test ipergeometrico. Se N è il numero di geni di riferimento, K quelli annotati al termine, n i DEG e k i DEG annotati al termine, la probabilità di osservare almeno k geni per caso è:

$$p = \Sigma_{i \geq k}\ C(K, i) \cdot C(N - K, n - i) / C(N, n)$$

Enrichr (Ma'ayan Lab, Icahn School of Medicine at Mount Sinai) confronta la lista con centinaia di librerie di gene set e per ogni termine riporta:

- **P-value** del test di Fisher e **Adjusted P-value** corretto con Benjamini–Hochberg;
- **Odds Ratio**: quanto il termine è sovra-rappresentato rispetto all'atteso;
- **Combined Score** = ln(p) × z, che combina il p-value con lo z-score della deviazione dal rango atteso (corregge la tendenza del test di Fisher a favorire i gene set grandi).

Si analizzano **separatamente** i geni up- e down-regolati: unire le due liste può diluire segnali opposti (es. proliferazione attivata e differenziamento perso). Le librerie scelte sono: **GO Biological Process** (processi biologici), **KEGG** e **Reactome** (pathway), **WikiPathways** (pathway curati dalla comunità).

14.2 Codice

Script R – Enrichr

```
## ---- 13. ANALISI DI ARRICCHIMENTO FUNZIONALE CON ENRICHR ------------------
# Richiede connessione internet (API https://maayanlab.cloud/Enrichr).
# In alternativa: caricare lista_UP.txt / lista_DOWN.txt sul sito web.
run_enrichment <- function(genes, label, dbs) {
  enr <- enrichR::enrichr(genes, dbs)
  for (db in names(enr)) {
    tab <- enr[[db]]
    if (is.null(tab) || nrow(tab) == 0) next
    tab <- tab[order(tab$Adjusted.P.value), ]
    write.table(tab, file.path(OUT_DIR,
                paste0("enrichr_", label, "_", db, ".txt")),
                sep = "\t", quote = FALSE, row.names = FALSE)
    top <- head(tab, 15)
    top$Term <- factor(substr(top$Term, 1, 60),
                       levels = rev(unique(substr(top$Term, 1, 60))))
    p <- ggplot(top, aes(-log10(Adjusted.P.value), Term,
                         size = Combined.Score)) +
      geom_point(colour = ifelse(label == "UP", "#C00000", "#2E75B6")) +
      labs(title = paste("EnrichR -", db, "- geni", label),
           x = "-log10(p-value aggiustato)", y = NULL) + theme_bw()
    save_plot(p, paste0("12_enrichr_", label, "_", db, ".png"), 10, 6)
  }
  invisible(enr)
}

if (requireNamespace("enrichR", quietly = TRUE)) {
  library(enrichR)            # il caricamento imposta le opzioni interne del pacchetto
  setEnrichrSite("Enrichr")
  dbs <- c("GO_Biological_Process_2023", "KEGG_2021_Human",
```

```
          "Reactome_2022", "WikiPathway_2023_Human")
  # verificare i nomi disponibili con: enrichR::listEnrichrDbs()
  enr_up   <- tryCatch(run_enrichment(out$Gene[out$Regulation == "UP"],
                                      "UP", dbs),
                       error = function(e) message("EnrichR UP: ", e$message))
  enr_down <- tryCatch(run_enrichment(out$Gene[out$Regulation == "DOWN"],
                                      "DOWN", dbs),
                       error = function(e) message("EnrichR DOWN: ", e$message))
} else {
  message("Pacchetto enrichR non installato: usare il sito web di Enrichr")
}
```

Spiegazione del codice

- `library(enrichR)` va eseguito prima di `setEnrichrSite`: è al caricamento del pacchetto che vengono impostati gli indirizzi del servizio web.
- `enrichR::enrichr(genes, dbs)` invia la lista di simboli all'API di Enrichr e restituisce una lista di data frame, uno per libreria.
- Per ogni libreria la tabella viene ordinata per p-value aggiustato e salvata; il **dot plot** mostra i 15 termini più significativi (asse x: −log10 p_adj; dimensione del punto: Combined Score).
- `tryCatch` evita che un errore di rete interrompa lo script; se il pacchetto non è installato si possono caricare manualmente i file `lista_UP.txt` / `lista_DOWN.txt` su https://maayanlab.cloud/Enrichr/.
- I nomi delle librerie cambiano con gli aggiornamenti di Enrichr: verificarli con `enrichR::listEnrichrDbs()`.

14.3 Risultati: geni up-regolati

Risultato: per i 580 geni up-regolati, nessun termine di **GO Biological Process** raggiunge FDR < 0,05 (il migliore ha FDR = 0,062). Risultano invece significativi **1 termine KEGG**, **10 termini Reactome** e **5 termini WikiPathways**.

Libreria	Termine	Overlap	FDR	Comb. score
KEGG	ECM-receptor interaction	11/88	0,012	48,4
Reactome	Extracellular Matrix Organization	27/291	$1{,}1 \times 10^{-4}$	56,5
Reactome	Collagen Formation	14/90	$1{,}4 \times 10^{-4}$	94,5
Reactome	Collagen Biosynthesis And Modifying Enzymes	11/67	$9{,}8 \times 10^{-4}$	84,4
Reactome	Assembly Of Collagen Fibrils	10/57	$1{,}1 \times 10^{-3}$	88,4
Reactome	Cell Cycle, Mitotic	33/523	$4{,}9 \times 10^{-3}$	24,5
WikiPathways	Gastric Cancer Network 1	8/27	$2{,}9 \times 10^{-4}$	203,4
WikiPathways	Metabolic Reprogramming In Colon Cancer	9/42	$5{,}9 \times 10^{-4}$	119,1
WikiPathways	Pancreatic Cancer Subtypes	9/48	$1{,}3 \times 10^{-3}$	91,6
GO BP (n.s.)	Positive Regulation Of Mitotic Sister Chromatid Separation	5/15	0,062	168,0
GO BP (n.s.)	Extracellular Structure Organization	12/109	0,062	39,9

Tabella – Principali termini arricchiti tra i geni up-regolati (n.s. = non significativo).

Emergono tre temi biologici coerenti:

- **Rimodellamento della matrice extracellulare**: collageni (COL1A1, COL1A2, COL3A1, COL5A2, COL10A1, COL11A1), metalloproteasi (MMP9, MMP11, MMP12), l'enzima PLOD2 e il chaperone SERPINH1, insieme a integrine e laminine (ITGA2, ITGA11, ITGB4, LAMB3) e a SPP1. Indicano la formazione di uno **stroma reattivo** (desmoplasia) prodotto dai fibroblasti associati al tumore, che favorisce invasione e migrazione.
- **Proliferazione**: geni del ciclo cellulare mitotico come TOP2A, AURKA, AURKB, BIRC5, CDC20, UBE2C, CCNB2, TPX2, MCM2 e MCM4, che riflettono l'alta attività proliferativa delle cellule tumorali.
- **Riprogrammazione metabolica**: LDHA, SLC2A1 (GLUT1), GAPDH, PFKP, HKDC1, SLC16A3 (MCT4), PSAT1 e SHMT2 indicano un aumento della glicolisi aerobica (**effetto Warburg**) e del metabolismo della serina. Compare inoltre ERBB2, recettore tirosin-chinasico oncogenico.

La significatività più debole degli up-regolati rispetto ai down-regolati è attesa: l'aumento di espressione nei tumori è eterogeneo tra pazienti, mentre la perdita dei geni del parenchima normale è sistematica. Il segnale proliferativo, diluito su centinaia di termini GO molto simili, perde significatività dopo la correzione FDR, ma è ben visibile nei termini Reactome e nei geni dei dot plot.

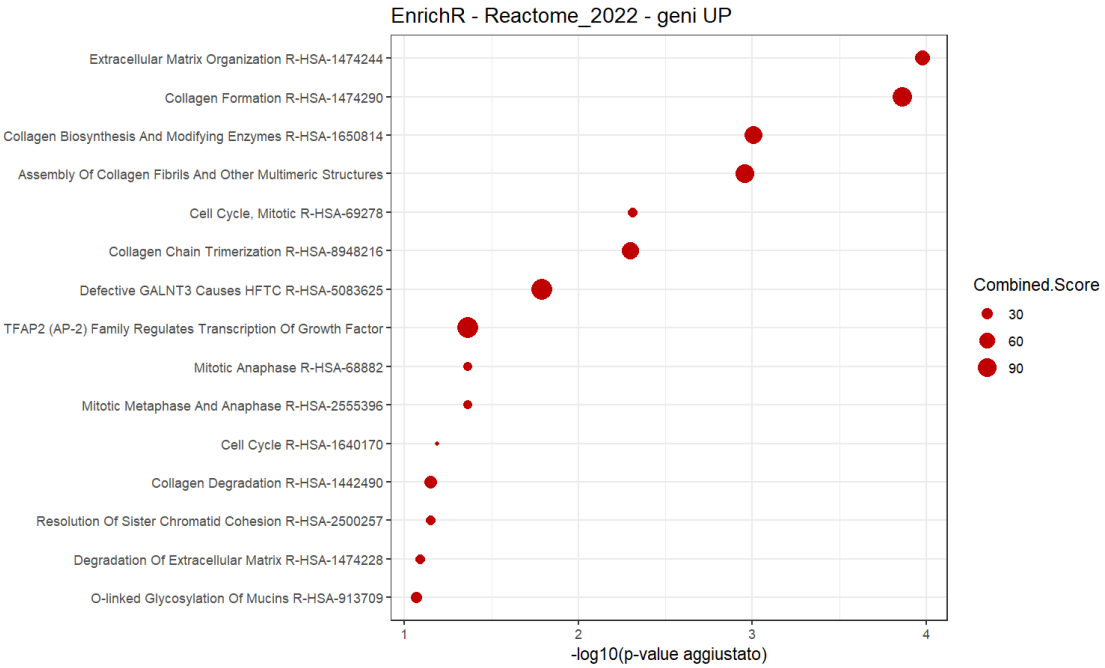


***Figura 13** – Enrichr – Reactome 2022, geni up-regolati (file: 12_enrichr_UP_Reactome_2022.png)*

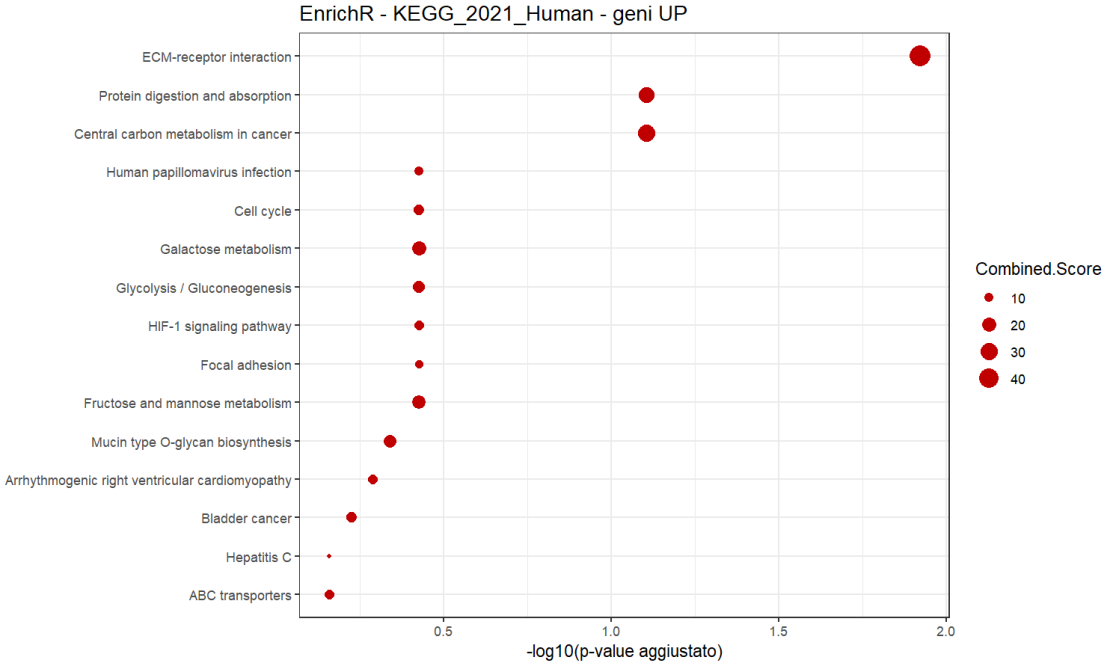


Figura 14 – *Enrichr – KEGG 2021, geni up-regolati (file: 12_enrichr_UP_KEGG_2021_Human.png)*

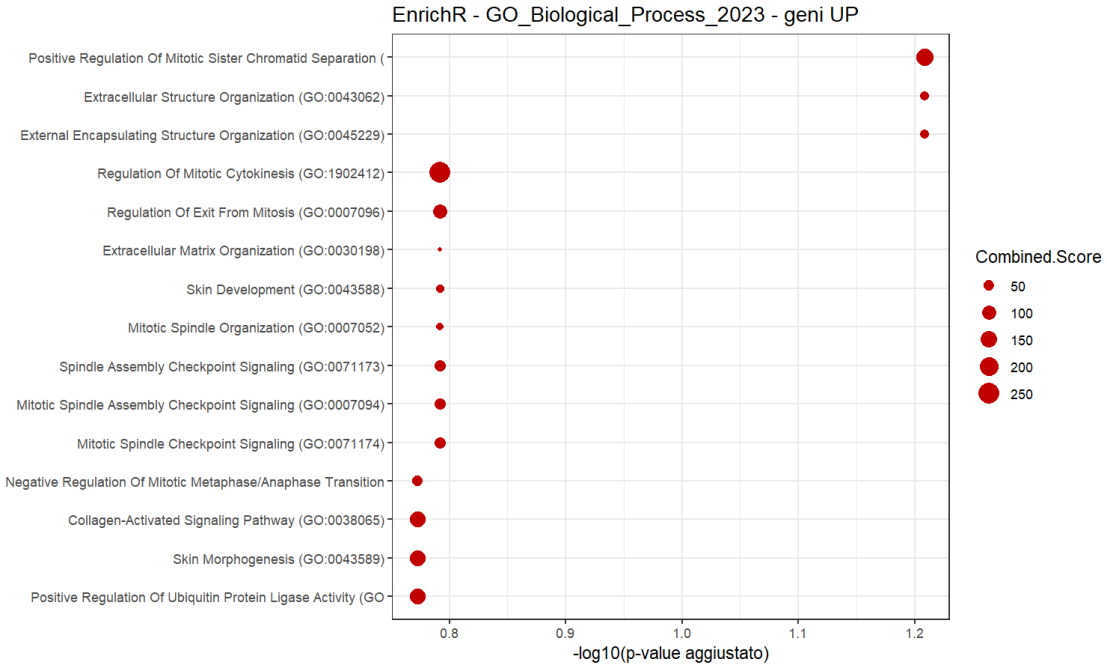


Figura 15 – *Enrichr – GO Biological Process 2023, geni up-regolati (file: 12_enrichr_UP_GO_Biological_Process_2023.png)*

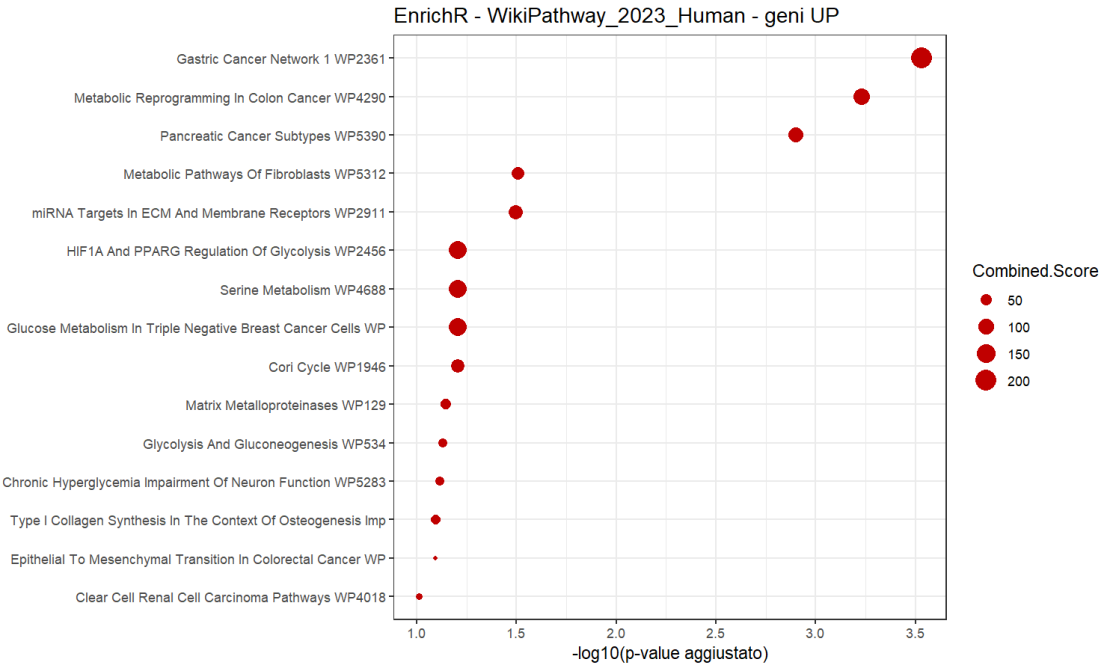


Figura 16 – *Enrichr – WikiPathways 2023, geni up-regolati (file: 12_enrichr_UP_WikiPathway_2023_Human.png)*

14.4 Risultati: geni down-regolati

Risultato: per i 784 geni down-regolati l'arricchimento è molto forte: **303 termini GO BP**, **54 KEGG**, **49 Reactome** e **60 WikiPathways** con FDR < 0,05.

Libreria	Termine	Overlap	FDR	Comb. score
GO BP	Cellular Response To Cytokine Stimulus	44/308	$3{,}1 \times 10^{-10}$	127,7
GO BP	Inflammatory Response	35/236	$1{,}8 \times 10^{-8}$	111,2
GO BP	Regulation Of Angiogenesis	31/205	$1{,}1 \times 10^{-7}$	103,3
GO BP	Cellular Response To TGF-beta Stimulus	19/96	$3{,}0 \times 10^{-6}$	118,1
KEGG	Complement and coagulation cascades	23/85	$3{,}0 \times 10^{-11}$	278,5
KEGG	Hematopoietic cell lineage	21/99	$2{,}7 \times 10^{-8}$	150,9
KEGG	Phagosome	25/152	$6{,}3 \times 10^{-8}$	101,8
Reactome	Complement Cascade	16/55	$2{,}1 \times 10^{-7}$	229,3
Reactome	Neutrophil Degranulation	48/468	$3{,}5 \times 10^{-7}$	59,8
Reactome	Diseases Associated With Surfactant Metabolism	6/9	$5{,}0 \times 10^{-5}$	747,0
Reactome	Metallothioneins Bind Metals	6/11	$2{,}2 \times 10^{-4}$	399,6
WikiPathways	Complement System	23/95	$8{,}0 \times 10^{-10}$	219,0
WikiPathways	Prostaglandin Synthesis And Regulation	11/45	$6{,}4 \times 10^{-5}$	111,4

Tabella – Principali termini arricchiti tra i geni down-regolati.

I termini dei geni down-regolati descrivono ciò che il tumore **perde** rispetto al polmone normale:

- **Immunità innata, complemento e infiammazione**: componenti del complemento (C1QA, C1QB, C1QC, C3, C7, CFD, CFH), chemochine (CCL2, CCL5, CCL14, CXCL2, CXCL12), recettori dei macrofagi

e dei neutrofili (CD14, CD36, MARCO, FPR1, C5AR1). Il polmone normale è ricco di macrofagi alveolari e di cellule immunitarie residenti, che il tumore sostituisce e in parte esclude.

- **Angiogenesi ed endotelio**: CDH5, TEK, ACVRL1, VWF, CA4, CLDN5, RAMP2 e AQP1 riflettono la fitta rete capillare alveolare, sostituita nel tumore da una vascolarizzazione meno organizzata. Il segnale delle emoglobine (HBA1, HBA2, HBB) ha la stessa origine: il polmone normale contiene molto più sangue capillare.
- **Funzione alveolare e surfattante**: SFTPA1, SFTPA2, SFTPB, SFTPC, SFTPD e ABCA3 sono marcatori degli pneumociti di tipo II e indicano la **perdita del differenziamento alveolare**.
- **Risposta al TGF-β** (TGFBR2, TGFBR3, SMAD6) e **metallotioneine** (MT1A, MT1E, MT1G, MT1M, MT1X, MT2A), entrambe descritte come ridotte in diversi tumori solidi.

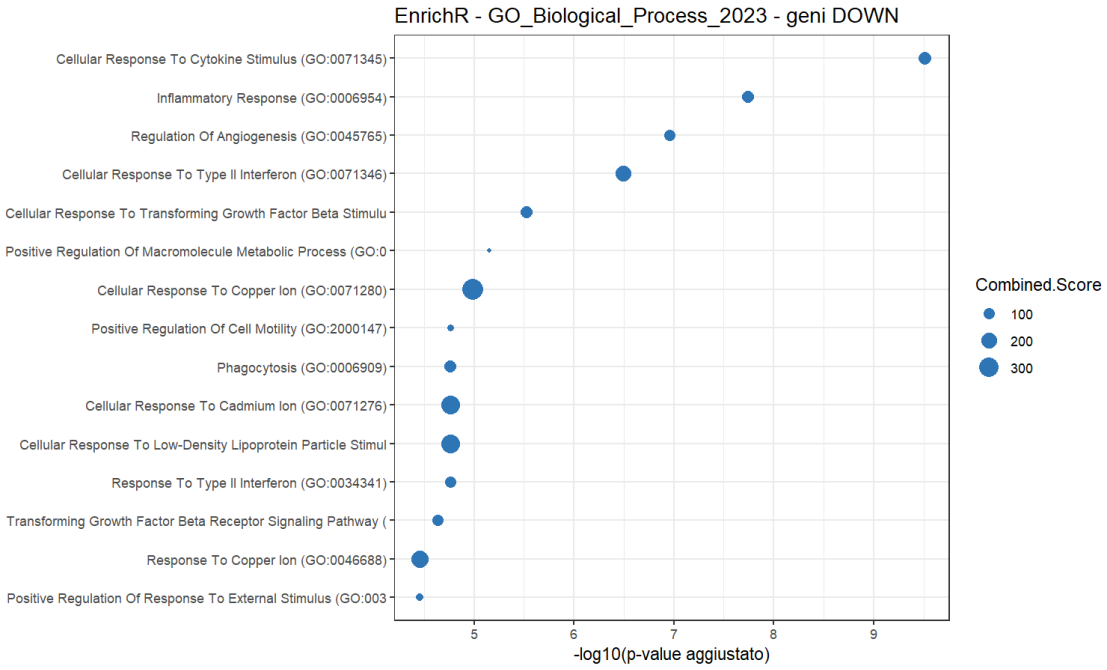


Figura 17 – *Enrichr – GO Biological Process 2023, geni down-regolati (file: 12_enrichr_DOWN_GO_Biological_Process_2023.png)*

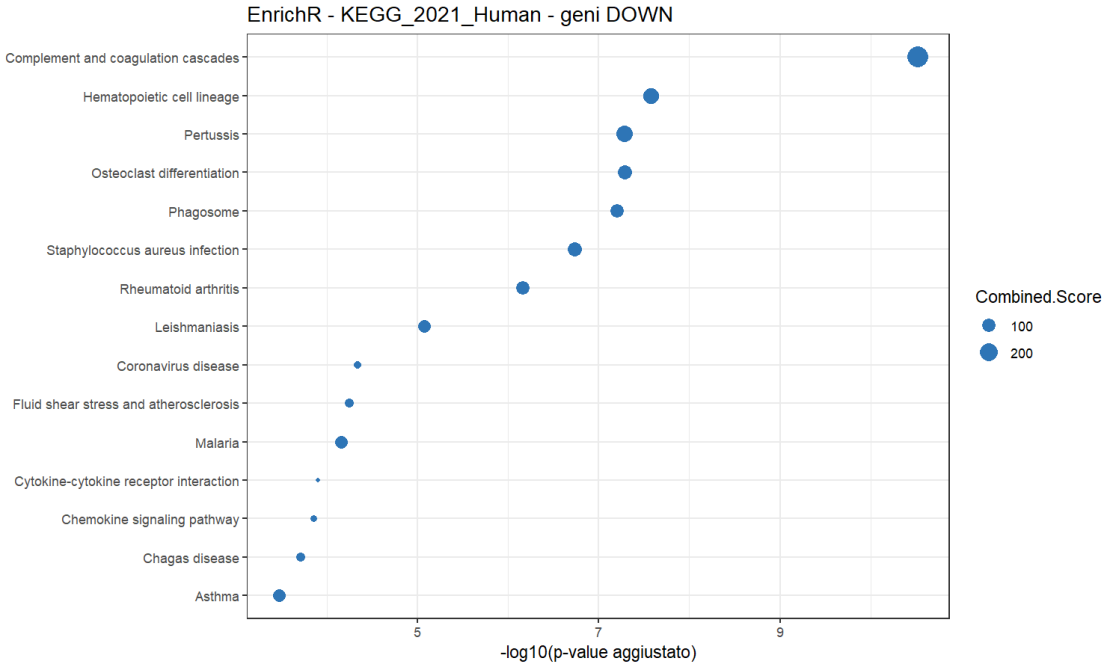


Figura 18 – *Enrichr – KEGG 2021, geni down-regolati (file: 12_enrichr_DOWN_KEGG_2021_Human.png)*

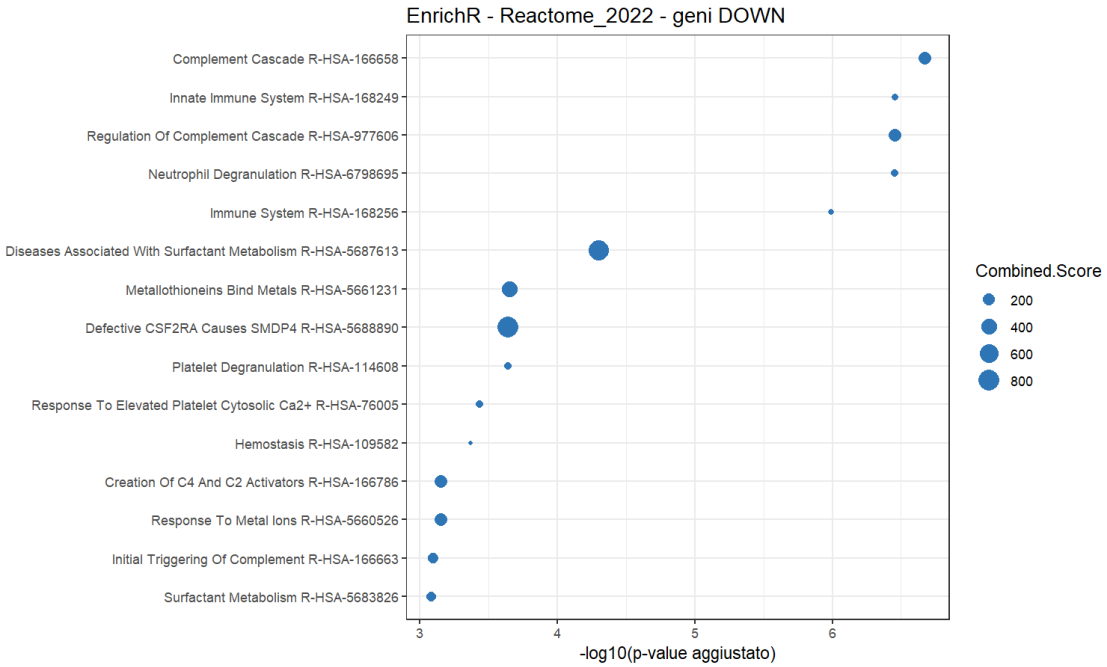


Figura 19 – *Enrichr – Reactome 2022, geni down-regolati (file: 12_enrichr_DOWN_Reactome_2022.png)*

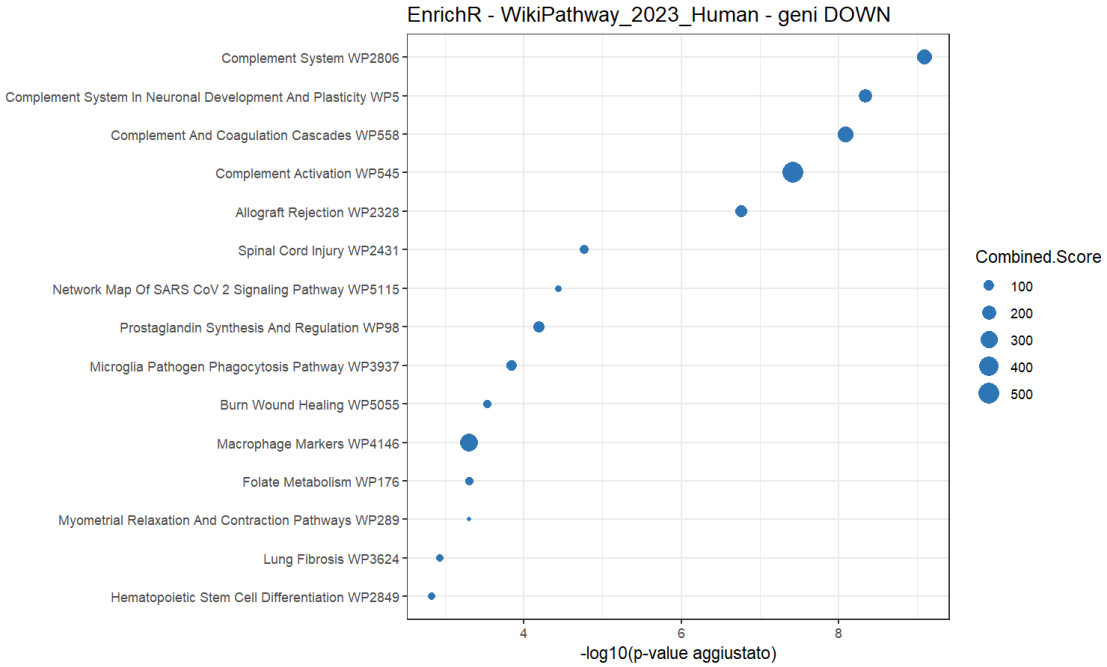


Figura 20 – *Enrichr – WikiPathways 2023, geni down-regolati (file: 12_enrichr_DOWN_WikiPathway_2023_Human.png)*

14.5 Confronto con la letteratura

Studi di espressione su adenocarcinoma polmonare vs polmone normale riportano in modo ricorrente i temi funzionali seguenti, che i risultati ottenuti confermano:

- **Geni up-regolati**: ciclo cellulare e divisione mitotica, organizzazione e degradazione della matrice extracellulare, interazione ECM-recettore. Confermati da KEGG (ECM-receptor interaction) e da Reactome (Extracellular Matrix Organization, Collagen Formation, Cell Cycle, Mitotic).
- **Geni down-regolati**: sviluppo vascolare e angiogenesi, funzioni dell'epitelio alveolare (surfattante), componente immunitaria del parenchima normale. Confermati da GO (Regulation Of Angiogenesis, Inflammatory Response), Reactome (Surfactant Metabolism) e da

KEGG/WikiPathways (complemento). La componente immunitaria e del complemento è risultata il segnale dominante, più marcato di quanto suggerito dalla sola letteratura sulle cellule tumorali: riflette la composizione cellulare del tessuto, aspetto discusso nella sezione 17.

Il quadro complessivo riflette tre fenomeni: la **proliferazione incontrollata** e la **riprogrammazione metabolica** delle cellule tumorali, la **perdita del differenziamento** alveolare e della componente vascolare e immunitaria del parenchima normale, e il **rimodellamento dello stroma** peritumorale.

15. Analisi di sensibilità: test t appaiato

Il disegno *matched-normal* fornisce per 57 pazienti sia il tumore sia il normale. Il **test t appaiato** lavora sulle differenze intra-paziente d*i = x*Tumor,i − x*Normal,i e verifica H0:* μd = 0 con:

$$t = \bar{d} / (s_d / \sqrt{n})$$

Poiché la variabilità tra individui viene rimossa, il test appaiato è in genere più potente. Confrontare i DEG ottenuti con i due test verifica la **robustezza** dei risultati rispetto alla scelta del test.

Script R – Test t appaiato

```
## ---- 14. ANALISI DI SENSIBILITA: TEST t APPAIATO -------------------------
# I campioni sono matched-normal: per i pazienti con entrambi i campioni si
# puo usare un test t appaiato, che elimina la variabilita tra individui.
iT <- match(paired_ids, clin$patient[case]);  colT <- colnames(gexpr)[case][iT]
iN <- match(paired_ids, clin$patient[ctrl]);  colN <- colnames(gexpr)[ctrl][iN]
d  <- gexpr[, colT] - gexpr[, colN]           # differenze intra-paziente
p_pair   <- apply(d, 1, function(x) t.test(x)$p.value)
padj_pair <- p.adjust(p_pair, method = "BH")
lfc_pair <- rowMeans(d)
deg_pair <- names(which(padj_pair < ALPHA & abs(lfc_pair) >= lfc_thr))
cat("DEG con test appaiato:", length(deg_pair),
    " - in comune con Welch:", length(intersect(deg_pair, deg$Gene)), "\n")
```

Spiegazione del codice

- `match` allinea, per ogni paziente in `paired_ids`, la colonna del tumore e quella del normale.
- `d` è la matrice delle differenze intra-paziente; il test t a un campione su `d` equivale al test appaiato.
- Si applicano le stesse soglie (FDR e |logFC|) e si conta l'intersezione con i DEG del test di Welch.

Risultato: il test t appaiato sui 57 pazienti con entrambi i campioni individua **1.364 DEG**, di cui **1.354 in comune** con il test di Welch (99,3% di sovrapposizione). I due elenchi differiscono per soli 10 geni per parte, tutti in prossimità delle soglie. I risultati sono quindi **robusti** rispetto alla scelta del test: la differenza tumore/normale è così marcata che l'eliminazione della variabilità tra pazienti non cambia in modo sostanziale l'elenco dei DEG.

16. Approfondimento con banche dati biologiche e ricerca bibliografica

16.1 Metodologia

Per i geni più rilevanti (top up/down per logFC, geni presenti nei termini arricchiti più significativi) si propone il seguente percorso:

1. **NCBI Gene / GeneCards / UniProt**: funzione molecolare, localizzazione cellulare, domini proteici, malattie associate.
2. **Human Protein Atlas** (sezione *Pathology*, carcinoma polmonare): conferma a livello proteico (immunoistochimica) e valore prognostico.
3. **GEPIA2** (o UALCAN): validazione su dati indipendenti RNA-seq di TCGA-LUAD vs tessuto normale TCGA/GTEx e curve di sopravvivenza.
4. **KEGG PATHWAY**: posizione dei geni nel pathway hsa05223 "Non-small cell lung cancer" e nei pathway emersi da Enrichr.
5. **STRING**: rete di interazione proteina-proteina dei DEG, per individuare moduli funzionali e geni hub.
6. **PubMed**: ricerca con query del tipo `"GENE"[Title/Abstract] AND "lung adenocarcinoma"[Title/Abstract]`, privilegiando studi su coorti indipendenti e meta-analisi.

16.2 Geni candidati noti in letteratura

La tabella confronta geni descritti in letteratura come differenzialmente espressi nell'adenocarcinoma polmonare con i risultati ottenuti (logFC e FDR dal file dei DEG o dalla tabella completa).

Gene	Atteso	Ottenuto	Funzione e rilevanza nel LUAD
SPP1	Up	UP, logFC 3,39 (1° up)	Osteopontina: glicoproteina secreta, ligando di integrine e CD44; promuove migrazione, invasione e metastasi; associata a prognosi sfavorevole.
MMP1 / MMP12	Up	MMP1 non DEG (logFC 0,81, FDR $1{,}1 \times 10^{-6}$); MMP12 UP, logFC 1,36	Metalloproteasi della matrice: degradano la ECM e favoriscono invasione tumorale. MMP1 è significativa ma sotto la soglia di fold-change.
COL11A1	Up	UP, logFC 1,85	Collagene prodotto dai fibroblasti associati al tumore; marcatore di stroma reattivo e invasività.
TOP2A	Up	UP, logFC 2,52	Topoisomerasi IIα, essenziale nella replicazione e divisione cellulare; marcatore di proliferazione.
CEACAM5	Up	UP, logFC 2,56	Antigene carcinoembrionario (CEA), marcatore sierico usato nel follow-up degli adenocarcinomi.
AGER	Down	DOWN, logFC −4,31	Recettore RAGE, molto espresso dagli pneumociti di tipo I; la sua perdita riflette la perdita dell'epitelio alveolare normale.
CLDN18	Down	DOWN, logFC −4,79 (1° down)	Claudina-18 (isoforma polmonare), componente delle giunzioni strette dell'epitelio alveolare; studi su modelli murini ne suggeriscono un ruolo di oncosoppressore.
SFTPC	Down	DOWN, logFC −3,68	Proteina C del surfattante, marcatore degli pneumociti di tipo II; ridotta nei tumori meno differenziati.
FABP4 / CA4	Down	DOWN, logFC −3,68 / −4,00	Geni di adipociti/macrofagi (FABP4) e dell'endotelio capillare alveolare (CA4); indicano la perdita del microambiente alveolare.

Gene	Atteso	Ottenuto	Funzione e rilevanza nel LUAD
TNNC1 / CLIC5	Down	DOWN, logFC −2,61 / −2,39	Geni tipici del tessuto polmonare normale, frequentemente tra i più down-regolati negli studi di espressione LUAD.

Su 12 geni attesi, 11 sono confermati come DEG con la direzione prevista; l'unica eccezione è MMP1, comunque significativamente aumentata nel tumore ma con un fold-change inferiore a 2. Ciò conferma la coerenza dell'analisi con la letteratura.

16.3 Schede dei geni principali ottenuti

Per i 5 geni più up-regolati e i 5 più down-regolati (esclusi gli identificativi LOC non caratterizzati) si riportano funzione e rilevanza, ricavate da NCBI Gene, UniProt e dalla letteratura. Per la validazione su dati indipendenti si suggerisce di consultare per ciascun gene GEPIA2 (TCGA-LUAD vs normale) e la sezione Pathology di Human Protein Atlas.

Gene	logFC	Funzione e interpretazione
SPP1	+3,39	Osteopontina; mediatore di adesione, migrazione e invasione tramite integrine e CD44. È tra i geni più consistentemente aumentati nel LUAD.
GCNT3	+3,22	Glicosiltransferasi della biosintesi delle O-glicoproteine di tipo mucinico; il suo aumento è coerente con la produzione di mucine (vedi MUC4, MUC5AC, MUC16 tra gli up-regolati).
XAGE1	+3,21	Antigene cancro-testicolo della famiglia GAGE: espresso nei tumori ma non nei tessuti adulti normali (eccetto il testicolo); studiato come bersaglio per immunoterapia nel LUAD.
SPINK1	+3,16	Inibitore della tripsina; nei tumori agisce anche come fattore di crescita autocrino con attività simile all'EGF.
MMP11	+3,00	Stromelisina-3, metalloproteasi prodotta dai fibroblasti stromali; marcatore del rimodellamento della matrice e dell'invasione.
CLDN18	−4,79	Claudina delle giunzioni strette dell'epitelio alveolare; la sua perdita indica dedifferenziamento e alterazione della barriera epiteliale.
AGER	−4,31	Recettore RAGE, marcatore degli pneumociti di tipo I; è anche il gene con significatività tra le più alte dell'intero dataset (FDR $6{,}8 \times 10^{-49}$).
MCEMP1	−4,24	Proteina di membrana di mastociti e monociti; riflette la riduzione della componente immunitaria residente.
SFTPA1	−4,16	Proteina A1 del surfattante (pneumociti di tipo II), con ruolo anche nell'immunità innata polmonare.
HBB	−4,15	Catena β dell'emoglobina: non è un gene regolato nel tumore, ma un indicatore del diverso contenuto di sangue tra parenchima normale e tessuto tumorale.

16.4 Collegamento con lo studio originale

Selamat et al. (2012) hanno integrato i dati di espressione con la metilazione del DNA, identificando 164 geni ipermetilati con riduzione dell'espressione e 57 geni ipometilati con aumento dell'espressione. I pathway coinvolti riguardano differenziamento cellulare, transizione epitelio-mesenchimale, segnalazione RAS e WNT e regolazione del ciclo cellulare. Lo stesso lavoro ha individuato **LGALS4** come unico gene significativamente ipermetilato e down-regolato nei fumatori rispetto ai non fumatori. È utile verificare se i DEG ottenuti includano geni appartenenti a queste categorie, in particolare quelli down-regolati che potrebbero essere silenziati epigeneticamente.

17. Discussione e limiti

La pipeline implementa un'analisi differenziale classica e completamente riproducibile. Alcune scelte meritano una discussione critica:

- **Test indipendente vs appaiato**: il test di Welch, richiesto dall'impostazione della traccia, ignora l'appaiamento dei campioni; il test appaiato (sezione 15) sfrutta il disegno e fornisce una verifica di robustezza.
- **Varianza per gene**: con n = 58 la stima della varianza di ogni gene è stabile; con campioni più piccoli sarebbe preferibile il t moderato di `limma`.
- **Eterogeneità tumorale**: i campioni tumorali contengono quote variabili di stroma e infiltrato immunitario; parte dei DEG può riflettere differenze di composizione cellulare più che di regolazione nelle cellule tumorali.
- **Composizione cellulare**: i risultati lo confermano direttamente. Tra i geni più down-regolati compaiono emoglobine, geni di macrofagi e del complemento e geni endoteliali, che indicano differenze nella proporzione di sangue, cellule immunitarie ed endotelio tra i due tessuti più che una regolazione trascrizionale nelle cellule epiteliali. Metodi di deconvoluzione (es. CIBERSORT, xCell) permetterebbero di stimare queste proporzioni.
- **Effetto batch**: i campioni sono stati ibridati in due batch, ma tumore e normale di ogni paziente appartengono allo stesso batch; il disegno è quindi bilanciato e il batch non introduce distorsioni nel confronto principale.
- **Soglia IQR**: il 10° percentile cade all'interno del picco delle sonde non espresse (sezione 8); una soglia più alta ridurrebbe il numero di test senza modificare l'elenco dei DEG.
- **Collasso delle sonde**: la scelta della sonda a IQR massimo è euristica; sonde dello stesso gene possono misurare isoforme diverse.
- **Soglie arbitrarie**: FC = 2 e FDR = 0,05 sono convenzioni; il numero di DEG dipende fortemente da esse, per cui i parametri sono definiti in testa allo script per facilitare analisi alternative.
- **Limiti dei microarray**: range dinamico limitato rispetto all'RNA-seq e possibile cross-ibridazione delle sonde.
- **Arricchimento**: l'ORA dipende dalla soglia di selezione dei DEG e dall'universo di riferimento; un'analisi GSEA sull'intera lista ordinata sarebbe un complemento naturale.

18. Conclusioni

È stata sviluppata in R una pipeline completa per l'identificazione e la caratterizzazione funzionale dei geni differenzialmente espressi tra adenocarcinoma polmonare e tessuto polmonare normale adiacente (GEO GSE32863, 58 vs 58 campioni). La pipeline comprende l'acquisizione dei dati da GEO, la descrizione della coorte, la pre-elaborazione (log2 e filtro IQR), il filtraggio statistico (logFC, test t di Welch, FDR di Benjamini–Hochberg), la visualizzazione (volcano plot, boxplot, heatmap con clustering gerarchico, PCA), l'esportazione dei risultati, l'arricchimento funzionale con Enrichr e un'analisi di sensibilità con test appaiato.

I risultati principali sono:

- i dati del Series Matrix sono risultati già log2-trasformati; dopo il filtro IQR e il collasso delle sonde sono stati testati **23.949 geni**;
- con $|\text{logFC}| \geq 1$ e FDR < 0,05 sono stati identificati **1.364 DEG** (**580 up-regolati**, **784 down-regolati**); il gene più up-regolato è **SPP1** (osteopontina) e il più down-regolato **CLDN18** (claudina-18);
- il clustering gerarchico separa **perfettamente** tumori e normali (58/58) e la PCA mostra due gruppi distinti lungo PC1, che spiega il 53,6% della varianza;
- il test t appaiato conferma il 99,3% dei DEG, a dimostrazione della robustezza dei risultati;
- l'arricchimento funzionale indica per i geni up-regolati **rimodellamento della matrice extracellulare e collagene**, **ciclo cellulare** e **riprogrammazione metabolica glicolitica**; per i down-regolati **complemento e immunità innata**, **angiogenesi** e **metabolismo del surfattante**;
- 11 geni su 12 descritti in letteratura come alterati nel LUAD sono confermati con la direzione attesa.

Nel complesso il profilo trascrizionale descrive un tumore proliferante e metabolicamente riprogrammato, circondato da uno stroma reattivo, che ha perso il differenziamento alveolare e la componente vascolare e immunitaria del parenchima normale, in accordo con le conoscenze sulla biologia dell'adenocarcinoma polmonare.

Appendice A – Script R completo

Lo script seguente è identico al file `analisi_DEG_GSE32863.R` allegato. Per eseguirlo: impostare come working directory una cartella vuota, installare i pacchetti (sezione 0) e lanciare `source("analisi_DEG_GSE32863.R")`. Tutti i grafici e le tabelle vengono creati nella cartella `output/`.

analisi_DEG_GSE32863.R

```
################################################################################
#  Elaborato di Elementi di Bioinformatica
#  Analisi di geni differenzialmente espressi (DEG)
#  Dataset : GEO GSE32863 - Adenocarcinoma polmonare (LUAD) vs tessuto
#            polmonare adiacente non tumorale (matched-normal)
#  Piattaforma: GPL6884 - Illumina HumanWG-6 v3.0 expression beadchip
#  Linguaggio : R (>= 4.2) + Bioconductor
################################################################################

## ---- 0. SETUP: pacchetti, parametri, cartelle --------------------------
# Installazione (da eseguire una sola volta):
# if (!requireNamespace("BiocManager", quietly = TRUE))
#   install.packages("BiocManager")
# BiocManager::install(c("GEOquery", "Biobase"))
# install.packages(c("ggplot2", "ggrepel", "pheatmap", "RColorBrewer",
#                    "matrixStats", "dplyr", "tidyr", "gridExtra", "enrichR"))

suppressPackageStartupMessages({
  library(GEOquery)      # download e parsing dei dati GEO
  library(Biobase)       # classe ExpressionSet (exprs, pData, fData)
  library(matrixStats)   # statistiche per riga veloci (rowIQRs, ...)
  library(ggplot2)       # grafici
  library(ggrepel)       # etichette non sovrapposte nel volcano plot
  library(pheatmap)      # heatmap con clustering gerarchico
  library(RColorBrewer)  # palette di colori
  library(dplyr)         # manipolazione tabelle
  library(tidyr)         # formato long/wide
  library(gridExtra)     # composizione di piu grafici
})

set.seed(123)                # riproducibilita
GSE_ID   <- "GSE32863"       # identificativo della serie GEO
DATA_DIR <- "data"           # cartella per i file scaricati
OUT_DIR  <- "output"         # cartella per grafici e risultati
dir.create(DATA_DIR, showWarnings = FALSE)
dir.create(OUT_DIR,  showWarnings = FALSE)

IQR_PERC <- 0.10   # percentile della distribuzione IQR sotto cui scartare
FC_THR   <- 2      # soglia di fold-change lineare -> |logFC| >= log2(2) = 1
ALPHA    <- 0.05   # soglia sul p-value aggiustato (FDR)
options(timeout = 1200)       # download lenti

# funzione di utilita per salvare i grafici ggplot in PNG
save_plot <- function(p, file, w = 8, h = 6) {
  ggsave(file.path(OUT_DIR, file), p, width = w, height = h, dpi = 150)
}

## ---- 1. DOWNLOAD E IMPORTAZIONE DEI DATI -------------------------------
```

```
# getGEO scarica il file "Series Matrix" (dati gia normalizzati dagli autori)
# e, con getGPL = TRUE, la tabella di annotazione della piattaforma GPL6884.
gse_list <- getGEO(GSE_ID, GSEMatrix = TRUE, getGPL = TRUE,
                   destdir = DATA_DIR)
length(gse_list)              # una sola piattaforma -> lista di lunghezza 1
gse  <- gse_list[[1]]         # oggetto ExpressionSet

expr <- exprs(gse)            # matrice sonde x campioni
pd   <- pData(gse)            # metadati dei campioni (phenoData)
fd   <- fData(gse)            # annotazione delle sonde (featureData)

cat("Sonde:", nrow(expr), " - Campioni:", ncol(expr), "\n")
print(expr[1:5, 1:4])         # prime righe/colonne della matrice
print(summary(as.vector(expr)))
print(colnames(fd))           # colonne disponibili per l'annotazione

## ---- 2. DESCRIZIONE DEL DATASET E CARATTERISTICHE CLINICHE -----------------
# Le caratteristiche cliniche sono nelle colonne di pData che terminano
# con ":ch1" (derivano dai campi "characteristics_ch1" di GEO).
clin_cols <- grep(":ch1$", colnames(pd), value = TRUE)
clin <- pd[, clin_cols, drop = FALSE]
colnames(clin) <- make.names(sub(":ch1$", "", clin_cols))

# Gruppo (Tumor / Normal) e identificativo del paziente ricavati dal titolo:
# "Lung adenocarcinoma 05L12_T (expression)"
# "Adjacent non-tumor lung tissue 05L12_N (expression)"
clin$group   <- ifelse(grepl("non-tumor", pd$title, ignore.case = TRUE),
                       "Normal", "Tumor")
clin$patient <- sub("^.* (\\S+)_[TN] \\(expression\\)$", "\\1", pd$title)
rownames(clin) <- rownames(pd)

print(table(clin$group))
str(clin)

# Le variabili cliniche si riferiscono al paziente: per non contarle due
# volte si usano solo i campioni tumorali (uno per paziente).
clin_pat <- clin[clin$group == "Tumor", ]

plot_var <- function(df, v) {
  x <- df[[v]]
  xn <- suppressWarnings(as.numeric(as.character(x)))
  if (sum(!is.na(xn)) > 0.8 * length(x) && length(unique(xn)) > 10) {
    # variabile numerica continua (es. eta) -> istogramma
    ggplot(data.frame(val = xn), aes(val)) +
      geom_histogram(bins = 15, fill = "#2E75B6", colour = "white") +
      labs(title = v, x = v, y = "N. pazienti") + theme_bw()
  } else {
    # variabile categorica -> barplot delle frequenze
    ggplot(data.frame(val = factor(x)), aes(val)) +
      geom_bar(fill = "#C55A11") +
      geom_text(stat = "count", aes(label = after_stat(count)), vjust = -0.3) +
      labs(title = v, x = NULL, y = "N. pazienti") + theme_bw() +
      theme(axis.text.x = element_text(angle = 30, hjust = 1))
  }
}
# variabili da rappresentare: escludo identificativi e colonne costanti
vars <- setdiff(colnames(clin_pat), c("group", "patient"))
vars <- vars[sapply(clin_pat[vars], function(z) {
  k <- length(unique(z)); k > 1 && k < nrow(clin_pat) })]
```

```
vars <- unique(c(vars, grep("age", colnames(clin_pat), value = TRUE,
                            ignore.case = TRUE)))
plots <- lapply(vars, function(v) plot_var(clin_pat, v))
if (length(plots) > 0) {
  g <- arrangeGrob(grobs = plots, ncol = 2)
  ggsave(file.path(OUT_DIR, "01_caratteristiche_cliniche.png"), g,
         width = 10, height = 3.5 * ceiling(length(plots) / 2), dpi = 150)
}
# barplot della numerosita dei gruppi
p_grp <- ggplot(clin, aes(group, fill = group)) + geom_bar() +
  geom_text(stat = "count", aes(label = after_stat(count)), vjust = -0.3) +
  scale_fill_manual(values = c(Normal = "#2E75B6", Tumor = "#C00000")) +
  labs(title = "Campioni per gruppo", x = NULL, y = "N. campioni") +
  theme_bw() + theme(legend.position = "none")
save_plot(p_grp, "02_campioni_per_gruppo.png", 5, 4)

## ---- 3. ESTRAZIONE DEI CAMPIONI DA CONFRONTARE: CASI vs CONTROLLI ----------
grp  <- factor(clin$group, levels = c("Normal", "Tumor"))  # Normal = riferimento
case <- grp == "Tumor"     # vettore logico dei casi
ctrl <- grp == "Normal"    # vettore logico dei controlli
cat("Casi (Tumor):", sum(case), " - Controlli (Normal):", sum(ctrl), "\n")

# coppie complete paziente tumore/normale (servono per l'analisi appaiata)
paired_ids <- intersect(clin$patient[case], clin$patient[ctrl])
cat("Pazienti con entrambi i campioni:", length(paired_ids), "\n")

## ---- 4. PRE-ELABORAZIONE (a): TRASFORMAZIONE log2(x + 1) -------------------
expr <- expr[complete.cases(expr), ]          # rimozione sonde con NA
if (min(expr) < 0) {                          # Illumina: possibili valori
  cat("Valori negativi:", sum(expr < 0), "-> posti a 0\n")  # negativi dopo
  expr[expr < 0] <- 0                         # sottrazione del background
}
if (max(expr) > 50) {
  logexpr <- log2(expr + 1)                   # dati in scala lineare
} else {
  # i dati risultano gia in scala log2: una seconda trasformazione
  # comprimerebbe artificialmente la variabilita
  message("Dati gia in scala logaritmica: trasformazione non riapplicata")
  logexpr <- expr
}
print(summary(as.vector(logexpr)))

# controllo qualita: distribuzione dei valori per campione
long <- data.frame(sample = rep(colnames(logexpr), each = nrow(logexpr)),
                   value  = as.vector(logexpr),
                   group  = rep(as.character(grp), each = nrow(logexpr)))
p_box <- ggplot(long, aes(sample, value, fill = group)) +
  geom_boxplot(outlier.size = 0.2, linewidth = 0.2) +
  scale_fill_manual(values = c(Normal = "#2E75B6", Tumor = "#C00000")) +
  labs(title = "Distribuzione dell'espressione log2 per campione",
       x = "Campioni", y = "log2(espressione + 1)") +
  theme_bw() + theme(axis.text.x = element_blank(),
                     axis.ticks.x = element_blank())
save_plot(p_box, "03_boxplot_campioni_log2.png", 12, 5)

## ---- 5. PRE-ELABORAZIONE (b): FILTRO SULL'INTERVALLO INTERQUARTILE ---------
iqr <- rowIQRs(logexpr)                       # IQR = Q3 - Q1 per ogni sonda
names(iqr) <- rownames(logexpr)
iqr_thr <- quantile(iqr, probs = IQR_PERC)   # soglia al percentile scelto
```

```
cat("Soglia IQR (", IQR_PERC * 100, "deg percentile):", round(iqr_thr, 4), "\n")

p_iqr <- ggplot(data.frame(iqr = iqr), aes(iqr)) +
  geom_histogram(bins = 100, fill = "grey60", colour = "grey30") +
  geom_vline(xintercept = iqr_thr, colour = "red", linewidth = 1,
             linetype = "dashed") +
  annotate("text", x = iqr_thr, y = Inf, vjust = 2, hjust = -0.05,
           label = paste0(IQR_PERC * 100, " percentile = ",
                          round(iqr_thr, 3)), colour = "red") +
  labs(title = "Distribuzione dell'IQR delle sonde",
       x = "IQR (scala log2)", y = "Frequenza") + theme_bw()
save_plot(p_iqr, "04_istogramma_IQR.png", 8, 5)

keep <- iqr > iqr_thr
filt <- logexpr[keep, ]
cat("Sonde prima:", nrow(logexpr), " - dopo filtro IQR:", nrow(filt), "\n")

## ---- 6. ANNOTAZIONE: DA SONDA A SIMBOLO GENICO ----------------------------
# Piu sonde possono mappare lo stesso gene: si conserva quella con IQR
# maggiore (la piu informativa); si scartano le sonde senza simbolo.
sym_col <- intersect(c("Symbol", "ILMN_Gene", "Gene symbol", "GENE_SYMBOL"),
                     colnames(fd))[1]
ann <- data.frame(probe  = rownames(filt),
                  symbol = as.character(fd[rownames(filt), sym_col]),
                  iqr    = iqr[rownames(filt)],
                  stringsAsFactors = FALSE)
ann <- ann[!is.na(ann$symbol) & ann$symbol != "", ]
ann <- ann[order(ann$symbol, -ann$iqr), ]
ann <- ann[!duplicated(ann$symbol), ]
gexpr <- filt[ann$probe, ]
rownames(gexpr) <- ann$symbol
cat("Geni unici dopo il collasso delle sonde:", nrow(gexpr), "\n")

## ---- 7. FILTRAGGIO: logFC, TEST t, CORREZIONE FDR --------------------------
# logFC: i dati sono in scala log2, quindi il rapporto diventa differenza
logFC <- rowMeans(gexpr[, case]) - rowMeans(gexpr[, ctrl])

# n = 58 per gruppo (> 30): per il teorema del limite centrale la media
# campionaria e approssimativamente normale -> test t a due campioni.
# Si usa la variante di Welch (varianze non assunte uguali).
pval <- apply(gexpr, 1, function(x)
  t.test(x[case], x[ctrl], var.equal = FALSE)$p.value)

# correzione per test multipli: False Discovery Rate di Benjamini-Hochberg
padj <- p.adjust(pval, method = "BH")

res <- data.frame(Gene = rownames(gexpr), logFC = logFC,
                  AveExpr = rowMeans(gexpr), P.Value = pval,
                  adj.P.Val = padj, stringsAsFactors = FALSE)
res <- res[order(res$adj.P.Val), ]
print(head(res, 10))
cat("Geni con FDR <", ALPHA, ":", sum(res$adj.P.Val < ALPHA), "\n")

# istogramma dei p-value grezzi (diagnostica: picco vicino a 0 = segnale)
p_ph <- ggplot(res, aes(P.Value)) +
  geom_histogram(bins = 50, fill = "#7F7F7F", colour = "white") +
  labs(title = "Distribuzione dei p-value (test t di Welch)",
       x = "p-value", y = "Frequenza") + theme_bw()
save_plot(p_ph, "05_istogramma_pvalue.png", 7, 4.5)
```

```
## ---- 8. VOLCANO PLOT E APPLICAZIONE DELLE SOGLIE ---------------------------
lfc_thr <- log2(FC_THR)      # soglia sul logFC (1 se FC_THR = 2)
res$Regulation <- "NS"
res$Regulation[res$logFC >=  lfc_thr & res$adj.P.Val < ALPHA] <- "Up"
res$Regulation[res$logFC <= -lfc_thr & res$adj.P.Val < ALPHA] <- "Down"
print(table(res$Regulation))

top_lab <- rbind(head(res[res$Regulation == "Up", ][
                   order(-res$logFC[res$Regulation == "Up"]), ], 10),
                 head(res[res$Regulation == "Down", ][
                   order(res$logFC[res$Regulation == "Down"]), ], 10))

p_vol <- ggplot(res, aes(logFC, -log10(adj.P.Val), colour = Regulation)) +
  geom_point(size = 0.8, alpha = 0.6) +
  scale_colour_manual(values = c(Down = "#2E75B6", NS = "grey70",
                                 Up = "#C00000")) +
  geom_vline(xintercept = c(-lfc_thr, lfc_thr), linetype = "dashed") +
  geom_hline(yintercept = -log10(ALPHA), linetype = "dashed") +
  geom_text_repel(data = top_lab, aes(label = Gene), size = 3,
                  max.overlaps = 30, show.legend = FALSE) +
  labs(title = "Volcano plot - LUAD vs tessuto normale",
       x = "log2 Fold Change (Tumor - Normal)",
       y = "-log10(p-value aggiustato FDR)") + theme_bw()
save_plot(p_vol, "06_volcano_plot.png", 9, 7)

# sottoinsieme finale dei geni differenzialmente espressi (DEG)
deg <- res[res$Regulation != "NS", ]
deg <- deg[order(-abs(deg$logFC)), ]
cat("DEG totali:", nrow(deg), " - Up:", sum(deg$Regulation == "Up"),
    " - Down:", sum(deg$Regulation == "Down"), "\n")

## ---- 9. BOXPLOT DEL GENE PIU UP-REGOLATO E PIU DOWN-REGOLATO --------------
g_up   <- deg$Gene[which.max(deg$logFC)]
g_down <- deg$Gene[which.min(deg$logFC)]
cat("Gene piu up-regolato:", g_up, " - piu down-regolato:", g_down, "\n")

bx <- data.frame(Expression = c(gexpr[g_up, ], gexpr[g_down, ]),
                 Gene  = rep(c(paste(g_up, "(UP)"), paste(g_down, "(DOWN)")),
                             each = ncol(gexpr)),
                 Group = rep(as.character(grp), 2))
p_bx <- ggplot(bx, aes(Group, Expression, fill = Group)) +
  geom_boxplot(outlier.shape = NA, alpha = 0.8) +
  geom_jitter(width = 0.15, size = 0.8, alpha = 0.6) +
  facet_wrap(~ Gene, scales = "free_y") +
  scale_fill_manual(values = c(Normal = "#2E75B6", Tumor = "#C00000")) +
  labs(y = "log2(espressione + 1)", x = NULL,
       title = "Geni con la massima variazione") +
  theme_bw() + theme(legend.position = "none")
save_plot(p_bx, "07_boxplot_top_up_down.png", 9, 5)

## ---- 10. CLUSTERING E HEATMAP DEI DEG -------------------------------------
deg_mat <- gexpr[deg$Gene, ]
ann_col <- data.frame(Group = grp, row.names = colnames(deg_mat))
ann_colors <- list(Group = c(Normal = "#2E75B6", Tumor = "#C00000"))

# a) tutti i DEG: z-score per riga, distanza di correlazione, legame average
pheatmap(deg_mat, scale = "row",
         clustering_distance_rows = "correlation",
```

```
         clustering_distance_cols = "correlation",
         clustering_method = "average",
         color = colorRampPalette(rev(brewer.pal(11, "RdBu")))(100),
         breaks = seq(-3, 3, length.out = 101),
         annotation_col = ann_col, annotation_colors = ann_colors,
         show_rownames = nrow(deg_mat) <= 60, show_colnames = FALSE,
         main = paste("Heatmap dei", nrow(deg_mat), "DEG"),
         filename = file.path(OUT_DIR, "08_heatmap_DEG.png"),
         width = 10, height = 10)

# b) i 50 DEG con |logFC| maggiore, con i nomi dei geni leggibili
top50 <- head(deg$Gene, 50)
pheatmap(gexpr[top50, ], scale = "row",
         clustering_distance_rows = "correlation",
         clustering_distance_cols = "correlation",
         clustering_method = "average",
         color = colorRampPalette(rev(brewer.pal(11, "RdBu")))(100),
         breaks = seq(-3, 3, length.out = 101),
         annotation_col = ann_col, annotation_colors = ann_colors,
         show_colnames = FALSE, fontsize_row = 7,
         main = "Top 50 DEG per |logFC|",
         filename = file.path(OUT_DIR, "09_heatmap_top50.png"),
         width = 10, height = 10)

# raggruppamento dei campioni in 2 cluster e confronto con i gruppi reali
hc <- hclust(as.dist(1 - cor(deg_mat)), method = "average")
cl <- cutree(hc, k = 2)
print(table(Cluster = cl, Gruppo = grp))

## ---- 11. (OPZIONALE) PCA SUI DEG ----------------------------------------
pca <- prcomp(t(deg_mat), center = TRUE, scale. = TRUE)
var_exp <- 100 * pca$sdev^2 / sum(pca$sdev^2)
pc_df <- data.frame(PC1 = pca$x[, 1], PC2 = pca$x[, 2], Group = grp)
p_pca <- ggplot(pc_df, aes(PC1, PC2, colour = Group)) +
  geom_point(size = 2, alpha = 0.8) +
  stat_ellipse(level = 0.95, linetype = "dashed") +
  scale_colour_manual(values = c(Normal = "#2E75B6", Tumor = "#C00000")) +
  labs(title = "PCA dei campioni sui DEG",
       x = sprintf("PC1 (%.1f%%)", var_exp[1]),
       y = sprintf("PC2 (%.1f%%)", var_exp[2])) + theme_bw()
save_plot(p_pca, "10_PCA_DEG.png", 7, 5.5)

p_scree <- ggplot(data.frame(PC = 1:10, Var = var_exp[1:10]), aes(PC, Var)) +
  geom_col(fill = "#2E75B6") + geom_line() + geom_point() +
  scale_x_continuous(breaks = 1:10) +
  labs(title = "Scree plot", x = "Componente principale",
       y = "% varianza spiegata") + theme_bw()
save_plot(p_scree, "11_scree_plot.png", 6, 4)

## ---- 12. ESPORTAZIONE DEI RISULTATI -------------------------------------
out <- data.frame(Gene       = deg$Gene,
                  adj.P.Val  = signif(deg$adj.P.Val, 4),
                  logFC      = round(deg$logFC, 4),
                  Regulation = ifelse(deg$logFC > 0, "UP", "DOWN"))
write.table(out, file.path(OUT_DIR, "DEG_GSE32863.txt"),
            sep = "\t", quote = FALSE, row.names = FALSE)
# tabella completa (tutti i geni testati) e liste per EnrichR web
write.table(res, file.path(OUT_DIR, "risultati_completi_GSE32863.txt"),
            sep = "\t", quote = FALSE, row.names = FALSE)
```

```
writeLines(out$Gene[out$Regulation == "UP"],
           file.path(OUT_DIR, "lista_UP.txt"))
writeLines(out$Gene[out$Regulation == "DOWN"],
           file.path(OUT_DIR, "lista_DOWN.txt"))

## ---- 13. ANALISI DI ARRICCHIMENTO FUNZIONALE CON ENRICHR ------------------
# Richiede connessione internet (API https://maayanlab.cloud/Enrichr).
# In alternativa: caricare lista_UP.txt / lista_DOWN.txt sul sito web.
run_enrichment <- function(genes, label, dbs) {
  enr <- enrichR::enrichr(genes, dbs)
  for (db in names(enr)) {
    tab <- enr[[db]]
    if (is.null(tab) || nrow(tab) == 0) next
    tab <- tab[order(tab$Adjusted.P.value), ]
    write.table(tab, file.path(OUT_DIR,
                paste0("enrichr_", label, "_", db, ".txt")),
                sep = "\t", quote = FALSE, row.names = FALSE)
    top <- head(tab, 15)
    top$Term <- factor(substr(top$Term, 1, 60),
                       levels = rev(unique(substr(top$Term, 1, 60))))
    p <- ggplot(top, aes(-log10(Adjusted.P.value), Term,
                         size = Combined.Score)) +
      geom_point(colour = ifelse(label == "UP", "#C00000", "#2E75B6")) +
      labs(title = paste("EnrichR -", db, "- geni", label),
           x = "-log10(p-value aggiustato)", y = NULL) + theme_bw()
    save_plot(p, paste0("12_enrichr_", label, "_", db, ".png"), 10, 6)
  }
  invisible(enr)
}

if (requireNamespace("enrichR", quietly = TRUE)) {
  library(enrichR)            # il caricamento imposta le opzioni interne del pacchetto
  setEnrichrSite("Enrichr")
  dbs <- c("GO_Biological_Process_2023", "KEGG_2021_Human",
           "Reactome_2022", "WikiPathway_2023_Human")
  # verificare i nomi disponibili con: enrichR::listEnrichrDbs()
  enr_up   <- tryCatch(run_enrichment(out$Gene[out$Regulation == "UP"],
                                      "UP", dbs),
                       error = function(e) message("EnrichR UP: ", e$message))
  enr_down <- tryCatch(run_enrichment(out$Gene[out$Regulation == "DOWN"],
                                      "DOWN", dbs),
                       error = function(e) message("EnrichR DOWN: ", e$message))
} else {
  message("Pacchetto enrichR non installato: usare il sito web di Enrichr")
}

## ---- 14. ANALISI DI SENSIBILITA: TEST t APPAIATO --------------------------
# I campioni sono matched-normal: per i pazienti con entrambi i campioni si
# puo usare un test t appaiato, che elimina la variabilita tra individui.
iT <- match(paired_ids, clin$patient[case]);  colT <- colnames(gexpr)[case][iT]
iN <- match(paired_ids, clin$patient[ctrl]);  colN <- colnames(gexpr)[ctrl][iN]
d  <- gexpr[, colT] - gexpr[, colN]           # differenze intra-paziente
p_pair   <- apply(d, 1, function(x) t.test(x)$p.value)
padj_pair <- p.adjust(p_pair, method = "BH")
lfc_pair <- rowMeans(d)
deg_pair <- names(which(padj_pair < ALPHA & abs(lfc_pair) >= lfc_thr))
cat("DEG con test appaiato:", length(deg_pair),
    " - in comune con Welch:", length(intersect(deg_pair, deg$Gene)), "\n")
```

```
## ---- 15. RIPRODUCIBILITA ----------------------------------------------------
writeLines(capture.output(sessionInfo()),
           file.path(OUT_DIR, "sessionInfo.txt"))
cat("Analisi completata. Risultati in:", normalizePath(OUT_DIR), "\n")
```

Appendice B – File prodotti dallo script

File	Contenuto
01*caratteristiche*cliniche.png	Grafici delle variabili cliniche della coorte
02*campioni*per_gruppo.png	Numerosità dei gruppi
03*boxplot*campioni_log2.png	Controllo qualità delle distribuzioni per campione
04*istogramma*IQR.png	Distribuzione dell'IQR e soglia di filtro
05*istogramma*pvalue.png	Distribuzione dei p-value
06*volcano*plot.png	Volcano plot
07*boxplot*top*up*down.png	Boxplot del gene più up- e più down-regolato
08*heatmap*DEG.png, 09*heatmap*top50.png	Heatmap con clustering gerarchico
10*PCA*DEG.png, 11*scree*plot.png	PCA e varianza spiegata
12*enrichr*<UP|DOWN>_<libreria>.png	Dot plot dell'arricchimento funzionale
DEG_GSE32863.txt	File richiesto: Gene, adj.P.Val, logFC, UP/DOWN
risultati*completi*GSE32863.txt	Statistiche per tutti i geni testati
lista*UP.txt, lista*DOWN.txt	Liste di geni per Enrichr web
enrichr<*UP*|*DOWN*><libreria>.txt	Tabelle complete di Enrichr
sessionInfo.txt	Versioni di R e dei pacchetti (riproducibilità)

Appendice C – Ambiente software (sessionInfo)

Estratto del file `output/sessionInfo.txt` generato al termine dell'esecuzione.

Componente	Versione
R	4.6.1 (2026-06-24 ucrt)
Sistema operativo	Windows 11 x64, piattaforma x86_64-w64-mingw32
GEOquery	2.80.0
Biobase / BiocGenerics	2.72.0 / 0.58.1
limma (dipendenza di GEOquery)	3.68.5
matrixStats	1.5.0
ggplot2 / ggrepel	4.0.3 / 0.9.8
pheatmap / RColorBrewer	1.0.13 / 1.1-3
dplyr / tidyr / gridExtra	1.2.1 / 1.3.2 / 2.3.1
enrichR	3.4

Bibliografia

[1] Selamat SA, Chung BS, Girard L, et al. Genome-scale analysis of DNA methylation in lung adenocarcinoma and integration with mRNA expression. Genome Research. 2012;22(7):1197–1211.

[2] Edgar R, Domrachev M, Lash AE. Gene Expression Omnibus: NCBI gene expression and hybridization array data repository. Nucleic Acids Research. 2002;30(1):207–210.

[3] Davis S, Meltzer PS. GEOquery: a bridge between the Gene Expression Omnibus (GEO) and BioConductor. Bioinformatics. 2007;23(14):1846–1847.

[4] Huber W, Carey VJ, Gentleman R, et al. Orchestrating high-throughput genomic analysis with Bioconductor. Nature Methods. 2015;12(2):115–121.

[5] Welch BL. The generalization of "Student's" problem when several different population variances are involved. Biometrika. 1947;34(1–2):28–35.

[6] Benjamini Y, Hochberg Y. Controlling the false discovery rate: a practical and powerful approach to multiple testing. Journal of the Royal Statistical Society B. 1995;57(1):289–300.

[7] Bourgon R, Gentleman R, Huber W. Independent filtering increases detection power for high-throughput experiments. PNAS. 2010;107(21):9546–9551.

[8] Ritchie ME, Phipson B, Wu D, et al. limma powers differential expression analyses for RNA-sequencing and microarray studies. Nucleic Acids Research. 2015;43(7):e47.

[9] Chen EY, Tan CM, Kou Y, et al. Enrichr: interactive and collaborative HTML5 gene list enrichment analysis tool. BMC Bioinformatics. 2013;14:128.

[10] Kuleshov MV, Jones MR, Rouillard AD, et al. Enrichr: a comprehensive gene set enrichment analysis web server 2016 update. Nucleic Acids Research. 2016;44(W1):W90–W97.

[11] The Cancer Genome Atlas Research Network. Comprehensive molecular profiling of lung adenocarcinoma. Nature. 2014;511(7511):543–550.

[12] Tang Z, Kang B, Li C, Chen T, Zhang Z. GEPIA2: an enhanced web server for large-scale expression profiling and interactive analysis. Nucleic Acids Research. 2019;47(W1):W556–W560.

[13] Wickham H. ggplot2: Elegant Graphics for Data Analysis. Springer, 2016.

[14] Kolde R. pheatmap: Pretty Heatmaps. R package, CRAN.